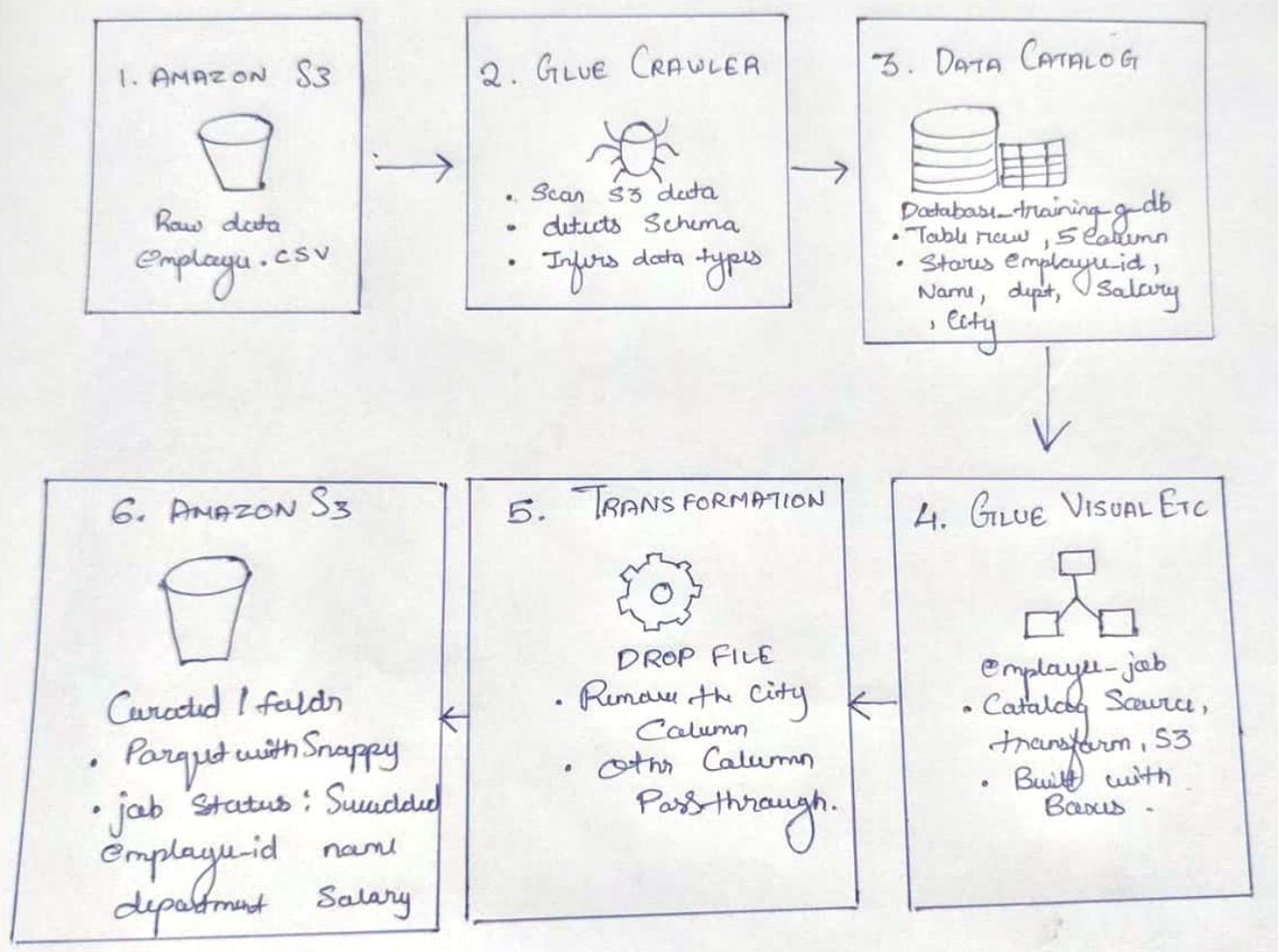
AWS CLOUD &
DATA ENGINEERING

Name	Adarsh Hajgulkar
USN / Roll Number	2JI23CS005
College Name	Jain College of Engineering, Belagavi
Batch	2023 - 2027
Date	11 oct 2026
Title	AWS Cloud & Data Engineering Practical Task

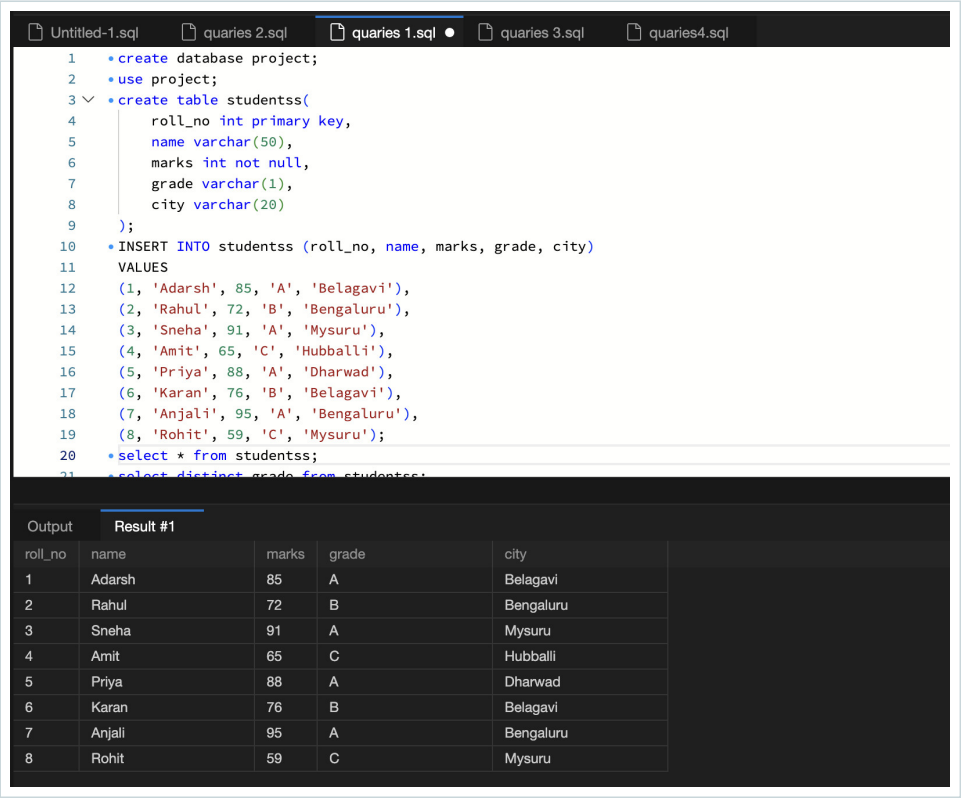
AWS Glue ETL - Complete workflow

Raw CSV in S3 is crawled, catalogued, transformed in Glue Visual ETL and saved as Parquet.



Q1 Create a Student Table

Creates the studentss table with columns for roll number, name, marks, grade and city, and inserts eight student records into it.



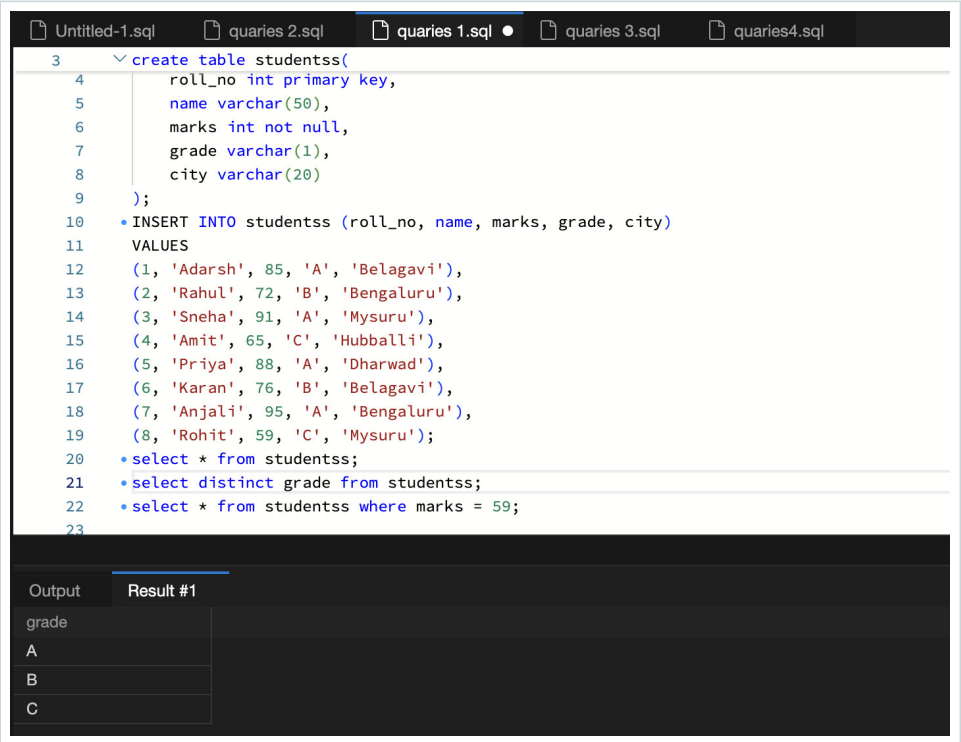
```sql
create database project;
use project;
create table studentss(
    roll_no int primary key,
    name varchar(50),
    marks int not null,
    grade varchar(1),
    city varchar(20)
);
INSERT INTO studentss (roll_no, name, marks, grade, city)
VALUES
(1, 'Adarsh', 85, 'A', 'Belagavi'),
(2, 'Rahul', 72, 'B', 'Bengaluru'),
(3, 'Sneha', 91, 'A', 'Mysuru'),
(4, 'Amit', 65, 'C', 'Hubballi'),
(5, 'Priya', 88, 'A', 'Dharwad'),
(6, 'Karan', 76, 'B', 'Belagavi'),
(7, 'Anjali', 95, 'A', 'Bengaluru'),
(8, 'Rohit', 59, 'C', 'Mysuru');
select * from studentss;
```

Output — Result #1

roll_no	name	marks	grade	city
1	Adarsh	85	A	Belagavi
2	Rahul	72	B	Bengaluru
3	Sneha	91	A	Mysuru
4	Amit	65	C	Hubballi
5	Priya	88	A	Dharwad
6	Karan	76	B	Belagavi
7	Anjali	95	A	Bengaluru
8	Rohit	59	C	Mysuru

Q2 Retrieve Distinct Grades

Displays all unique grades from the student table.



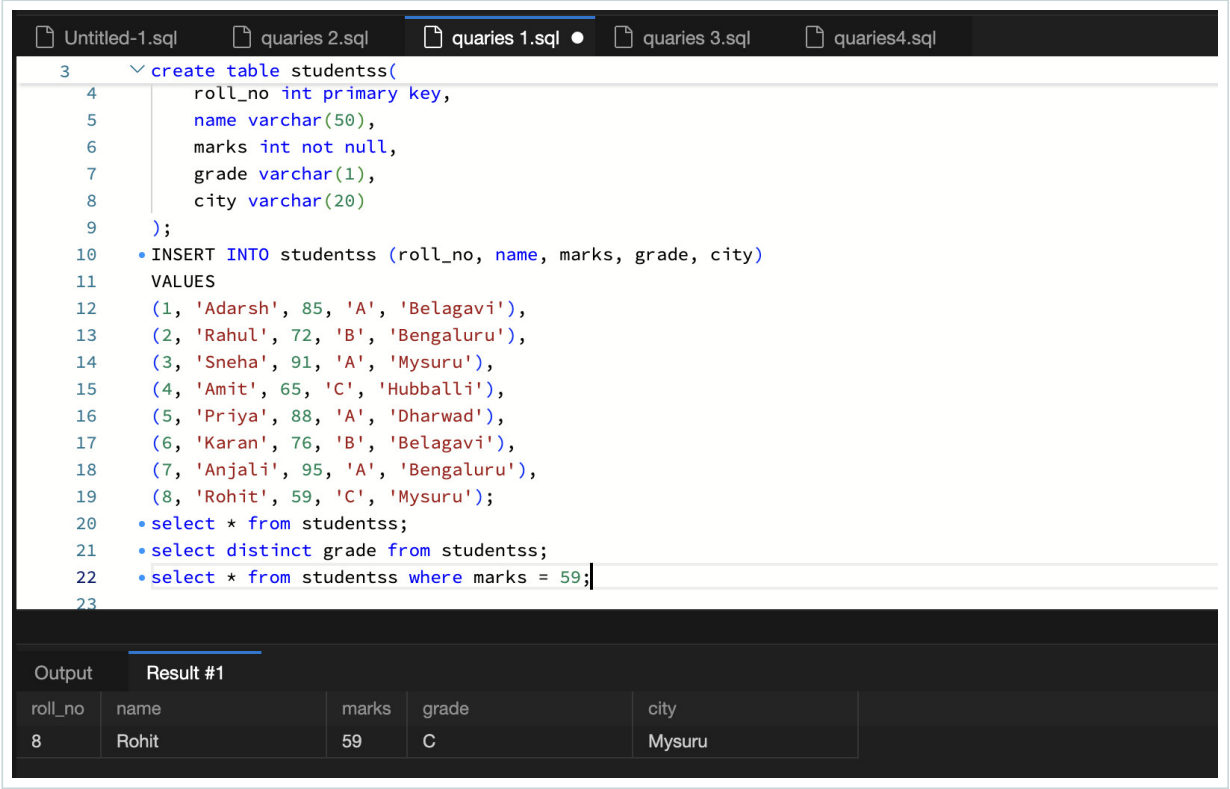
```sql
create table studentss(
    roll_no int primary key,
    name varchar(50),
    marks int not null,
    grade varchar(1),
    city varchar(20)
);
INSERT INTO studentss (roll_no, name, marks, grade, city)
VALUES
(1, 'Adarsh', 85, 'A', 'Belagavi'),
(2, 'Rahul', 72, 'B', 'Bengaluru'),
(3, 'Sneha', 91, 'A', 'Mysuru'),
(4, 'Amit', 65, 'C', 'Hubballi'),
(5, 'Priya', 88, 'A', 'Dharwad'),
(6, 'Karan', 76, 'B', 'Belagavi'),
(7, 'Anjali', 95, 'A', 'Bengaluru'),
(8, 'Rohit', 59, 'C', 'Mysuru');
select * from studentss;
select distinct grade from studentss;
select * from studentss where marks = 59;
```

Output — Result #1

grade
A
B
C

Q3 Retrieve Student Records Based on Marks

Displays the student record where marks are equal to 59.



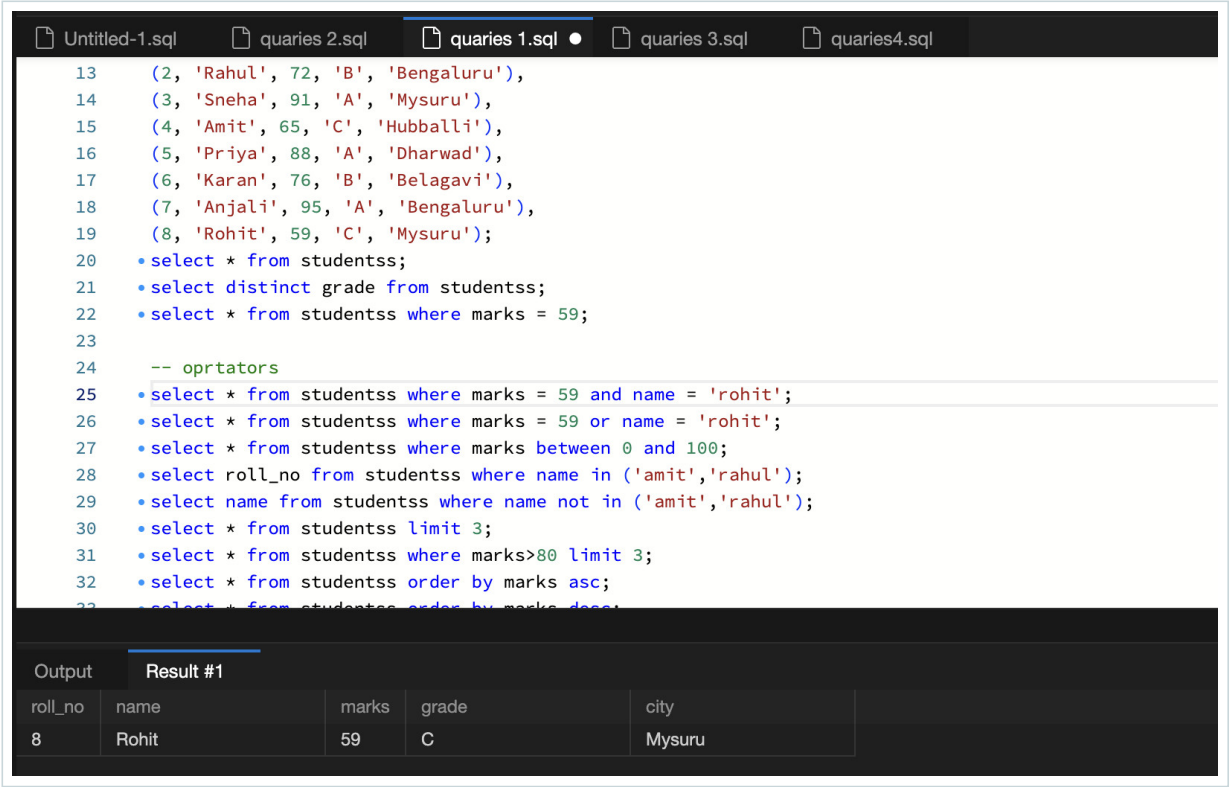
```sql
create table studentss(
    roll_no int primary key,
    name varchar(50),
    marks int not null,
    grade varchar(1),
    city varchar(20)
);
INSERT INTO studentss (roll_no, name, marks, grade, city)
VALUES
(1, 'Adarsh', 85, 'A', 'Belagavi'),
(2, 'Rahul', 72, 'B', 'Bengaluru'),
(3, 'Sneha', 91, 'A', 'Mysuru'),
(4, 'Amit', 65, 'C', 'Hubballi'),
(5, 'Priya', 88, 'A', 'Dharwad'),
(6, 'Karan', 76, 'B', 'Belagavi'),
(7, 'Anjali', 95, 'A', 'Bengaluru'),
(8, 'Rohit', 59, 'C', 'Mysuru');
select * from studentss;
select distinct grade from studentss;
select * from studentss where marks = 59;
```

Output — Result #1

roll_no	name	marks	grade	city
8	Rohit	59	C	Mysuru

Q4 Filter Records Using the AND Operator

Retrieves records where marks are 59 and the student's name is Rohit.



```sql
(2, 'Rahul', 72, 'B', 'Bengaluru'),
(3, 'Sneha', 91, 'A', 'Mysuru'),
(4, 'Amit', 65, 'C', 'Hubballi'),
(5, 'Priya', 88, 'A', 'Dharwad'),
(6, 'Karan', 76, 'B', 'Belagavi'),
(7, 'Anjali', 95, 'A', 'Bengaluru'),
(8, 'Rohit', 59, 'C', 'Mysuru');
select * from studentss;
select distinct grade from studentss;
select * from studentss where marks = 59;

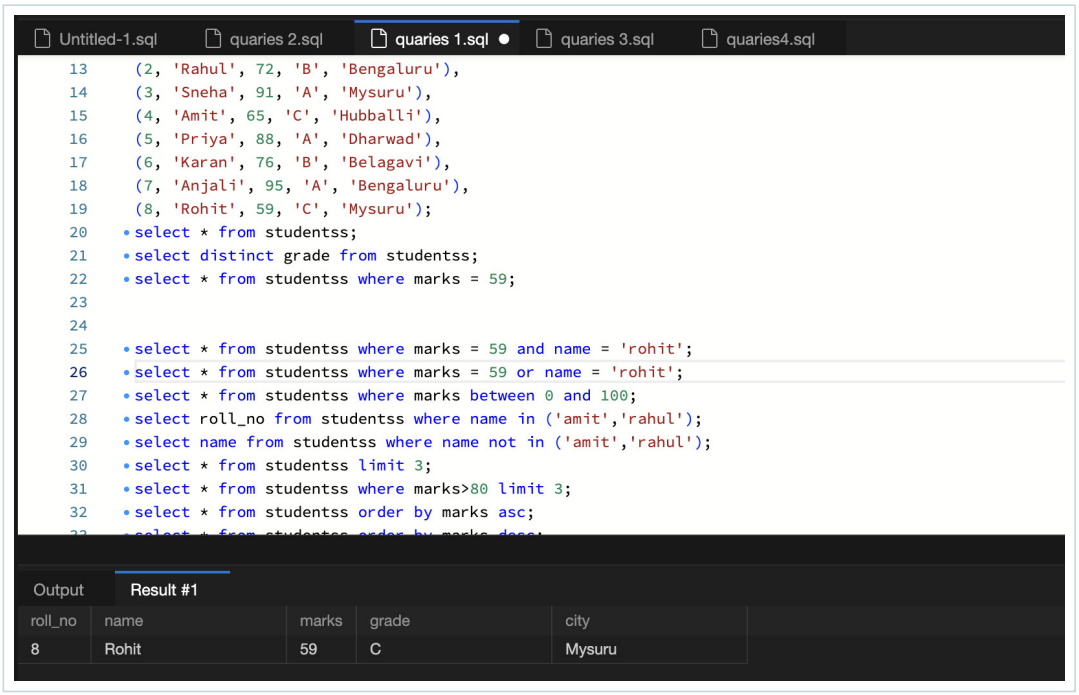
-- oprtators
select * from studentss where marks = 59 and name = 'rohit';
select * from studentss where marks = 59 or name = 'rohit';
select * from studentss where marks between 0 and 100;
select roll_no from studentss where name in ('amit','rahul');
select name from studentss where name not in ('amit','rahul');
select * from studentss limit 3;
select * from studentss where marks>80 limit 3;
select * from studentss order by marks asc;
```

Output — Result #1

roll_no	name	marks	grade	city
8	Rohit	59	C	Mysuru

Q5 Filter Records Using the OR Operator

Retrieves records where marks are 59 or the student's name is Rohit.



```
Untitled-1.sql | quaries 2.sql | quaries 1.sql | quaries 3.sql | quaries4.sql

    (2, 'Rahul', 72, 'B', 'Bengaluru'),
    (3, 'Sneha', 91, 'A', 'Mysuru'),
    (4, 'Amit', 65, 'C', 'Hubballi'),
    (5, 'Priya', 88, 'A', 'Dharwad'),
    (6, 'Karan', 76, 'B', 'Belagavi'),
    (7, 'Anjali', 95, 'A', 'Bengaluru'),
    (8, 'Rohit', 59, 'C', 'Mysuru');
select * from studentss;
select distinct grade from studentss;
select * from studentss where marks = 59;

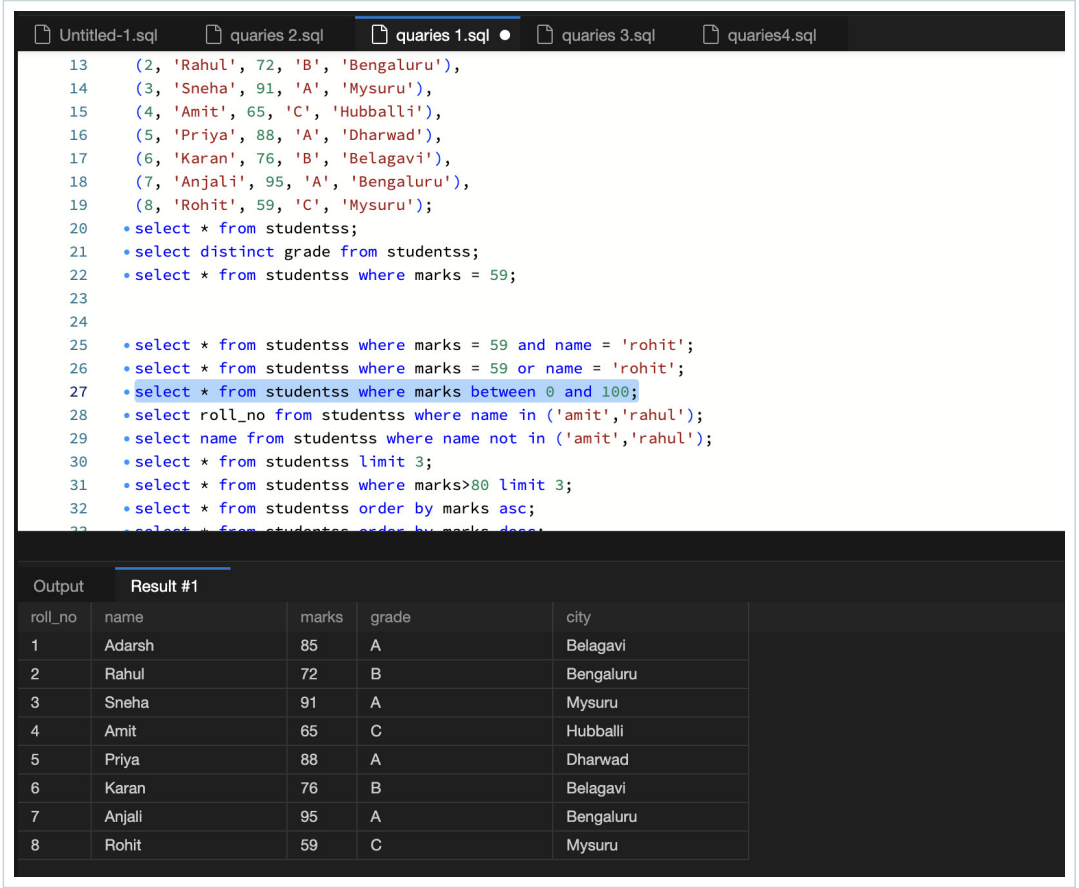

select * from studentss where marks = 59 and name = 'rohit';
select * from studentss where marks = 59 or name = 'rohit';
select * from studentss where marks between 0 and 100;
select roll_no from studentss where name in ('amit','rahul');
select name from studentss where name not in ('amit','rahul');
select * from studentss limit 3;
select * from studentss where marks>80 limit 3;
select * from studentss order by marks asc;
```

Output | Result #1

roll_no	name	marks	grade	city
8	Rohit	59	C	Mysuru

Q6 Retrieve Records Using the BETWEEN Operator

Displays students whose marks are between 0 and 100.



```
Untitled-1.sql | quaries 2.sql | quaries 1.sql | quaries 3.sql | quaries4.sql

    (2, 'Rahul', 72, 'B', 'Bengaluru'),
    (3, 'Sneha', 91, 'A', 'Mysuru'),
    (4, 'Amit', 65, 'C', 'Hubballi'),
    (5, 'Priya', 88, 'A', 'Dharwad'),
    (6, 'Karan', 76, 'B', 'Belagavi'),
    (7, 'Anjali', 95, 'A', 'Bengaluru'),
    (8, 'Rohit', 59, 'C', 'Mysuru');
select * from studentss;
select distinct grade from studentss;
select * from studentss where marks = 59;

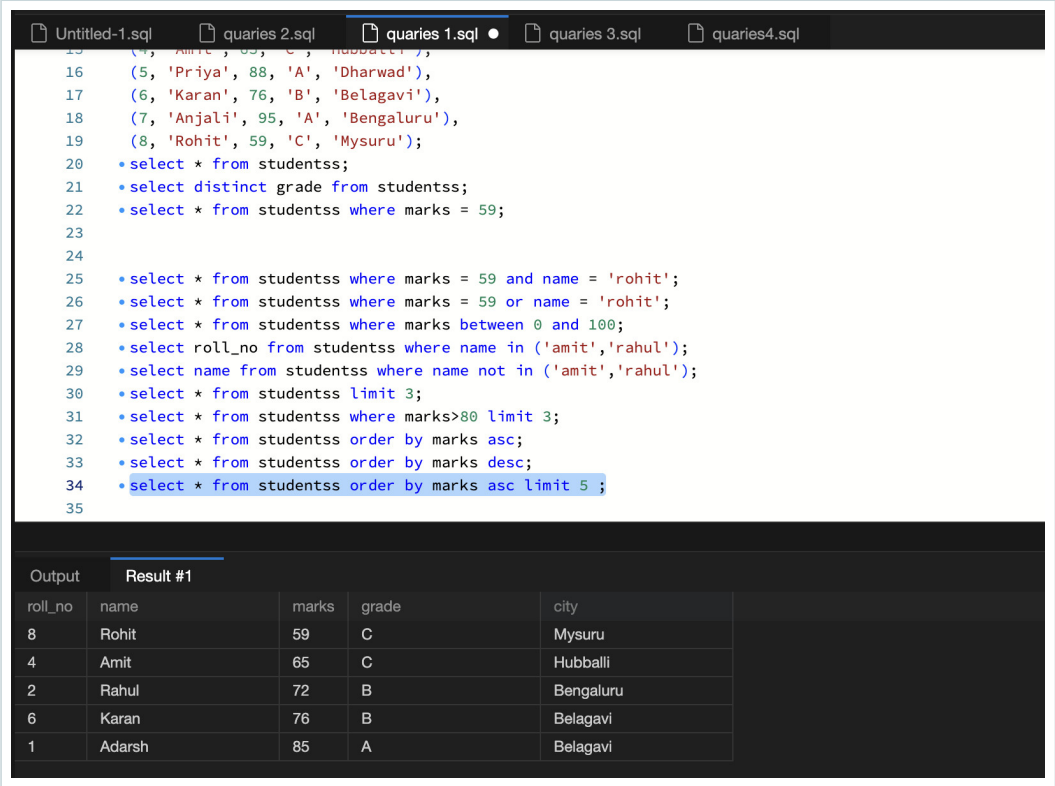

select * from studentss where marks = 59 and name = 'rohit';
select * from studentss where marks = 59 or name = 'rohit';
select * from studentss where marks between 0 and 100;
select roll_no from studentss where name in ('amit','rahul');
select name from studentss where name not in ('amit','rahul');
select * from studentss limit 3;
select * from studentss where marks>80 limit 3;
select * from studentss order by marks asc;
```

Output | Result #1

roll_no	name	marks	grade	city
1	Adarsh	85	A	Belagavi
2	Rahul	72	B	Bengaluru
3	Sneha	91	A	Mysuru
4	Amit	65	C	Hubballi
5	Priya	88	A	Dharwad
6	Karan	76	B	Belagavi
7	Anjali	95	A	Bengaluru
8	Rohit	59	C	Mysuru

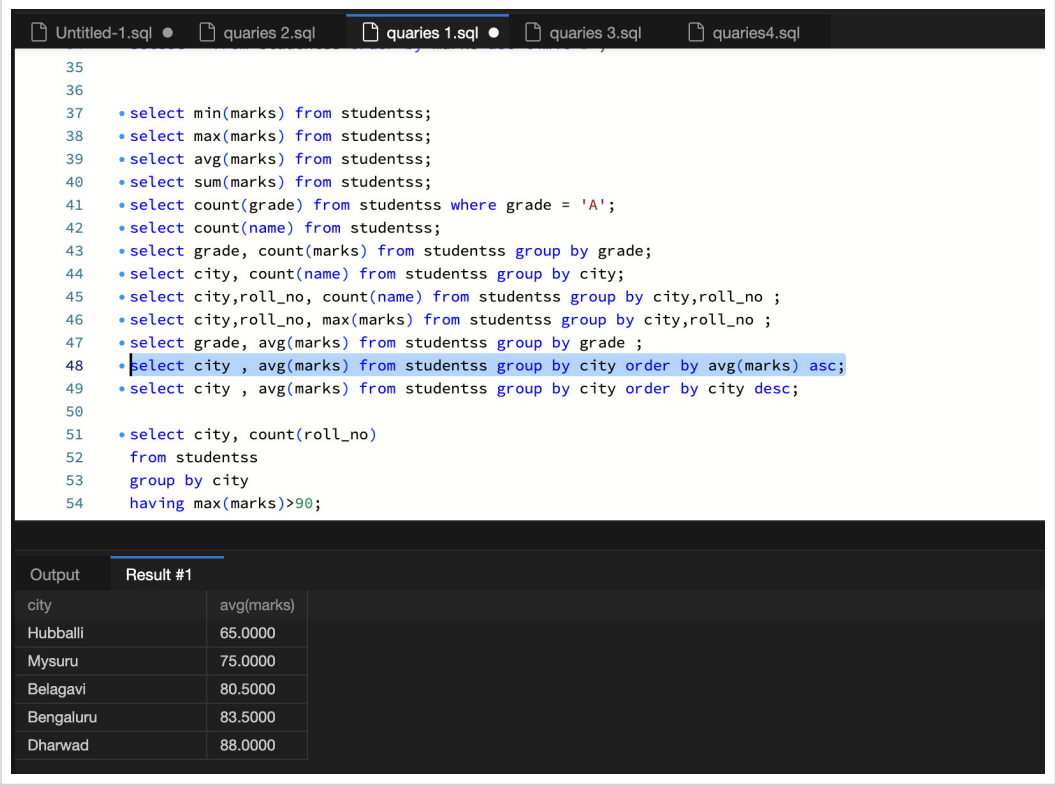
Q7 Retrieve the Five Students with the Lowest Marks

Displays the first five students after sorting their marks in ascending order.



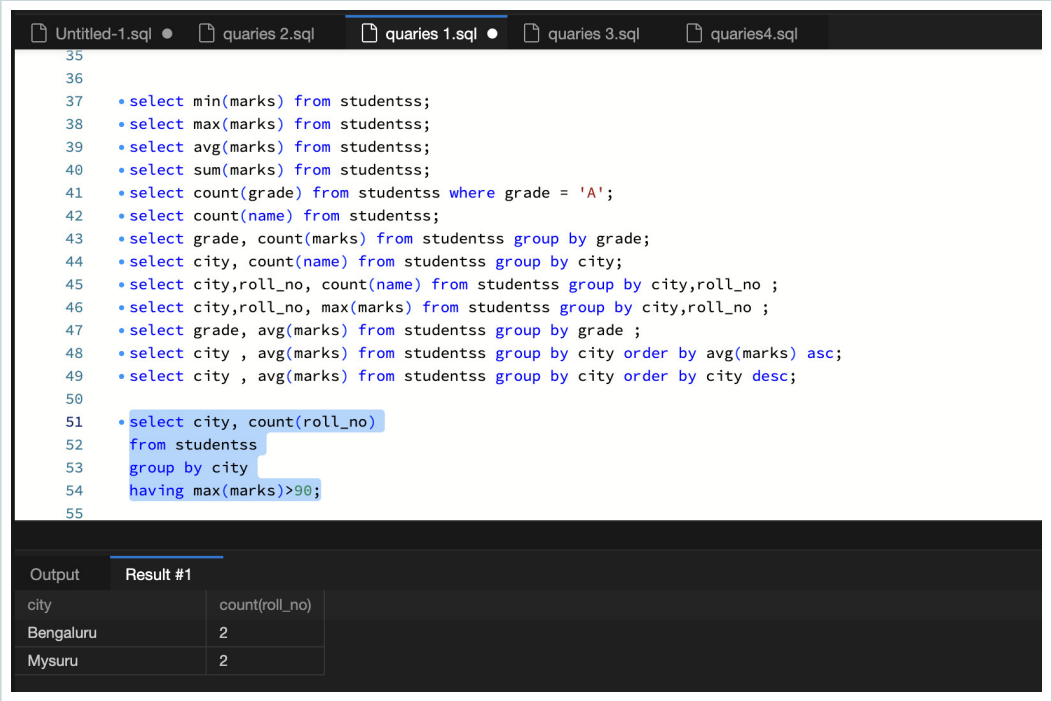
Q8 Calculate Average Marks by City in Ascending Order

Displays the average marks for each city, sorted from the lowest average to the highest.



Q9 Find Cities Where the Maximum Marks Exceed 90

Groups students by city and displays only cities where the highest marks exceed 90.



```
Untitled-1.sql ● | quaries 2.sql | quaries 1.sql ● | quaries 3.sql | quaries4.sql

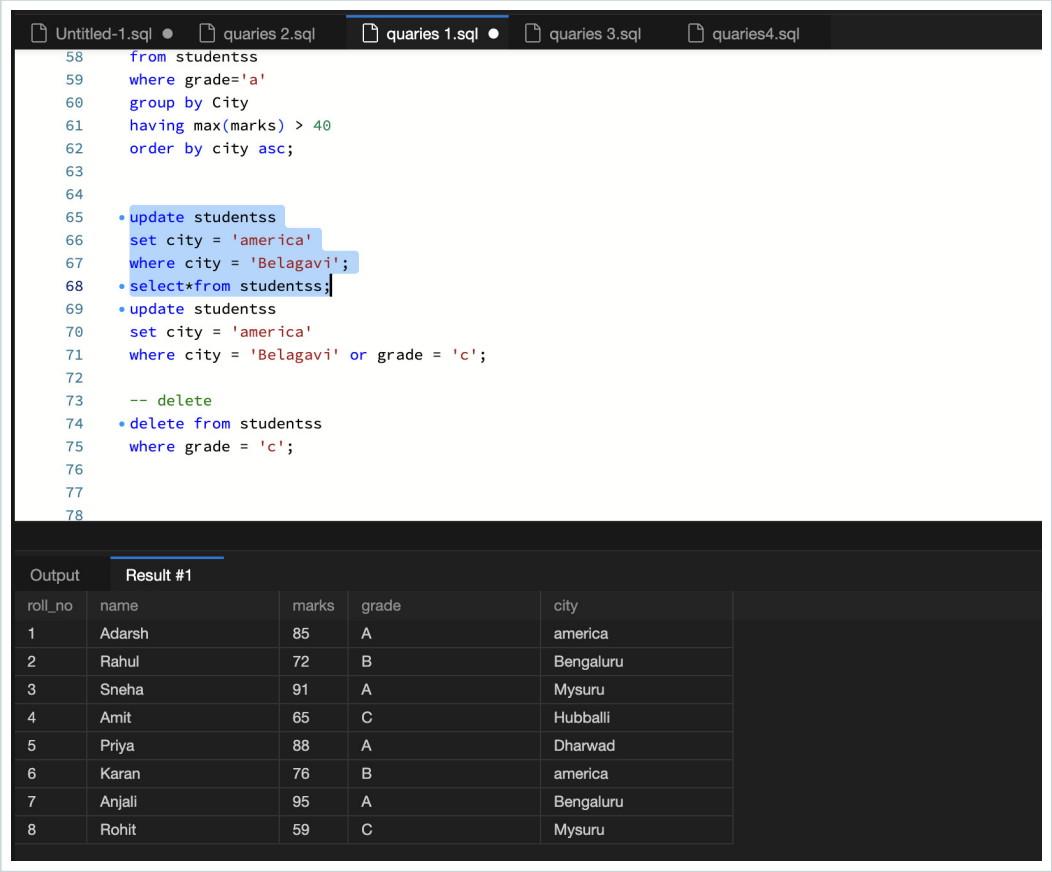
35
36
37  select min(marks) from studentss;
38  select max(marks) from studentss;
39  select avg(marks) from studentss;
40  select sum(marks) from studentss;
41  select count(grade) from studentss where grade = 'A';
42  select count(name) from studentss;
43  select grade, count(marks) from studentss group by grade;
44  select city, count(name) from studentss group by city;
45  select city,roll_no, count(name) from studentss group by city,roll_no ;
46  select city,roll_no, max(marks) from studentss group by city,roll_no ;
47  select grade, avg(marks) from studentss group by grade ;
48  select city , avg(marks) from studentss group by city order by avg(marks) asc;
49  select city , avg(marks) from studentss group by city order by city desc;
50
51  select city, count(roll_no)
52   from studentss
53   group by city
54   having max(marks)>90;
55
```

Output | Result #1

city	count(roll_no)
Bengaluru	2
Mysuru	2

Q10 Update the City Name for Belagavi Students

Changes the city from Belagavi to America for students whose city is Belagavi.



```
Untitled-1.sql ● | quaries 2.sql | quaries 1.sql ● | quaries 3.sql | quaries4.sql

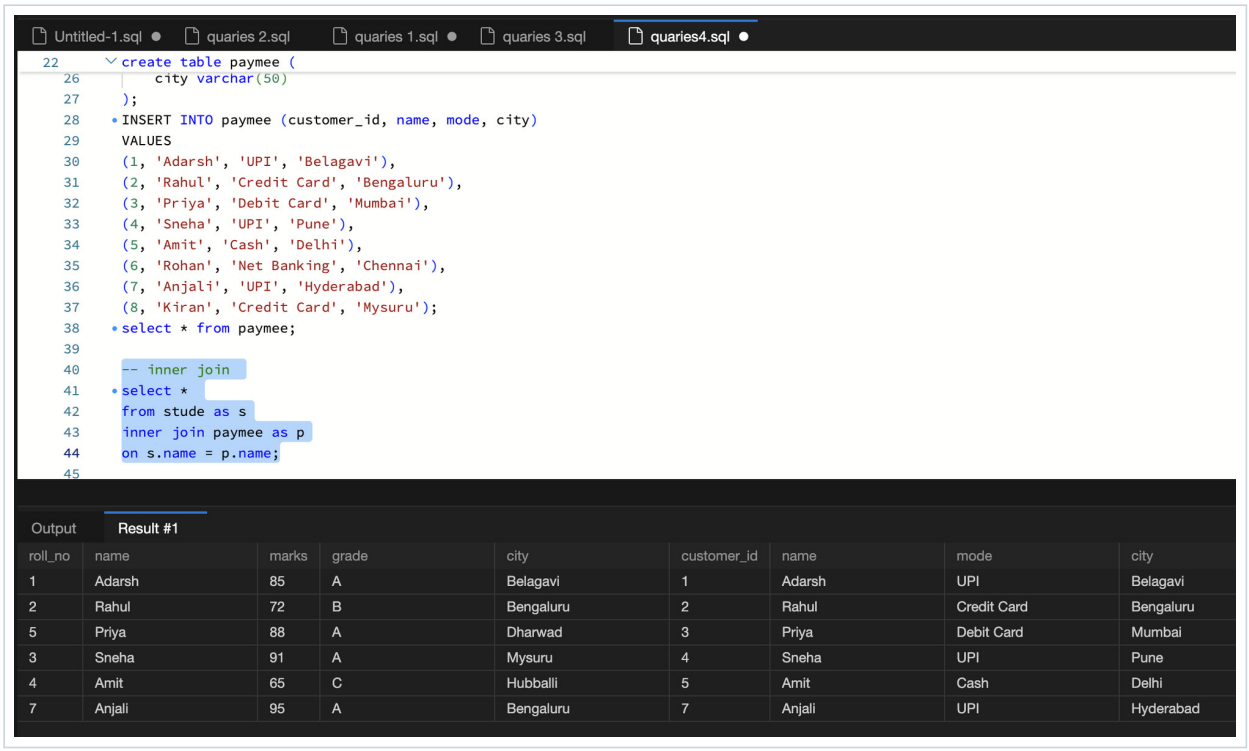
58   from studentss
59   where grade='a'
60   group by City
61   having max(marks) > 40
62   order by city asc;
63
64
65  update studentss
66   set city = 'america'
67   where city = 'Belagavi';
68  select*from studentss;
69  update studentss
70   set city = 'america'
71   where city = 'Belagavi' or grade = 'c';
72
73   -- delete
74  delete from studentss
75   where grade = 'c';
76
77
78
```

Output | Result #1

roll_no	name	marks	grade	city
1	Adarsh	85	A	america
2	Rahul	72	B	Bengaluru
3	Sneha	91	A	Mysuru
4	Amit	65	C	Hubballi
5	Priya	88	A	Dharwad
6	Karan	76	B	america
7	Anjali	95	A	Bengaluru
8	Rohit	59	C	Mysuru

Q11 INNER JOIN – Retrieve Matching Student and Payment Records

Displays records where the student name matches the customer name in both tables.



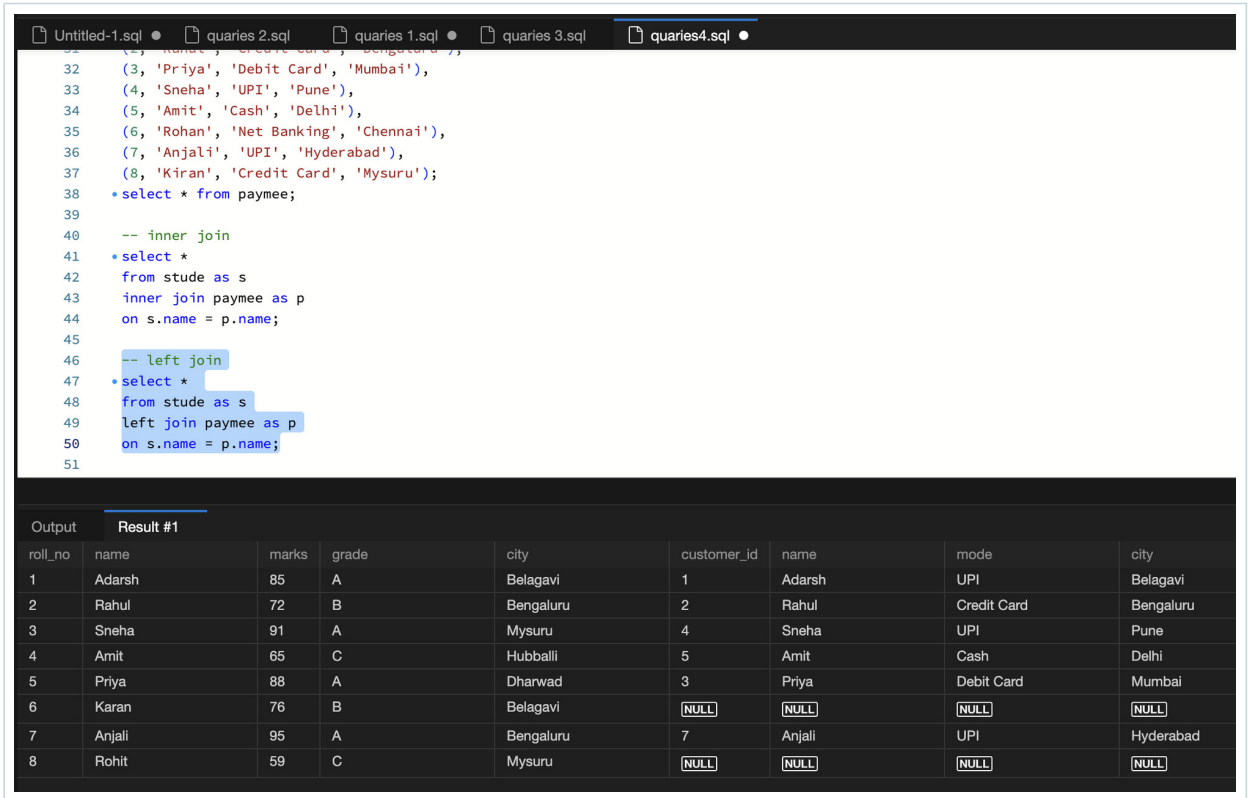
```
Untitled-1.sql ● quaries 2.sql quaries 1.sql ● quaries 3.sql quaries4.sql ●
22  create table paymee (
26      city varchar(50)
27  );
28  INSERT INTO paymee (customer_id, name, mode, city)
29  VALUES
30  (1, 'Adarsh', 'UPI', 'Belagavi'),
31  (2, 'Rahul', 'Credit Card', 'Bengaluru'),
32  (3, 'Priya', 'Debit Card', 'Mumbai'),
33  (4, 'Sneha', 'UPI', 'Pune'),
34  (5, 'Amit', 'Cash', 'Delhi'),
35  (6, 'Rohan', 'Net Banking', 'Chennai'),
36  (7, 'Anjali', 'UPI', 'Hyderabad'),
37  (8, 'Kiran', 'Credit Card', 'Mysuru');
38  select * from paymee;
39
40  -- inner join
41  select *
42  from stude as s
43  inner join paymee as p
44  on s.name = p.name;
45
```

Output | Result #1

roll_no	name	marks	grade	city	customer_id	name	mode	city
1	Adarsh	85	A	Belagavi	1	Adarsh	UPI	Belagavi
2	Rahul	72	B	Bengaluru	2	Rahul	Credit Card	Bengaluru
5	Priya	88	A	Dharwad	3	Priya	Debit Card	Mumbai
3	Sneha	91	A	Mysuru	4	Sneha	UPI	Pune
4	Amit	65	C	Hubballi	5	Amit	Cash	Delhi
7	Anjali	95	A	Bengaluru	7	Anjali	UPI	Hyderabad

Q12 LEFT JOIN – Retrieve All Students and Matching Payment Records

Displays all student records along with matching payment records. Students without matching payments are also included.



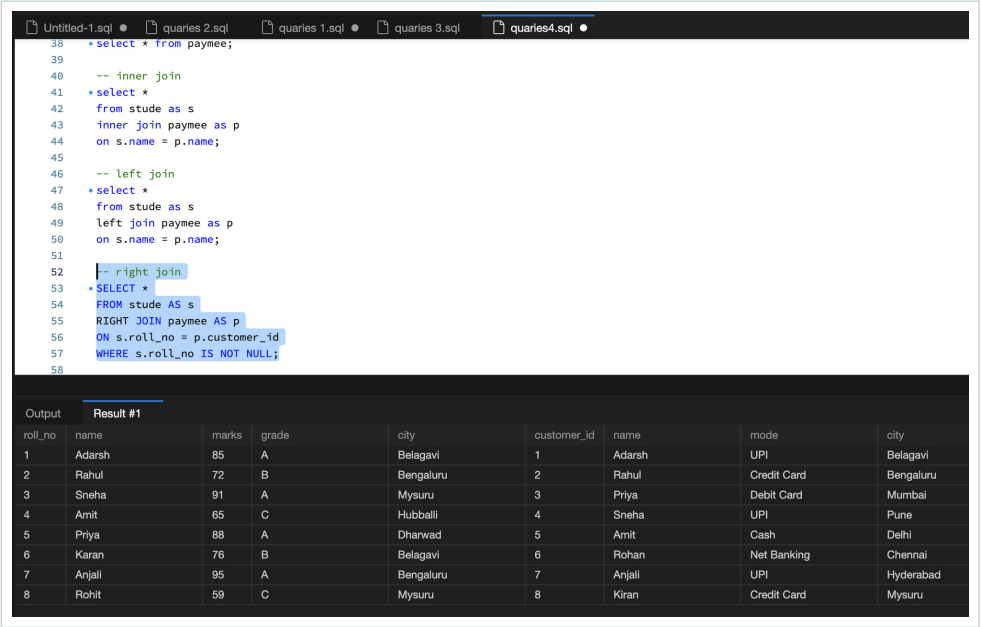
```
Untitled-1.sql ● quaries 2.sql quaries 1.sql ● quaries 3.sql quaries4.sql ●
32  (3, 'Priya', 'Debit Card', 'Mumbai'),
33  (4, 'Sneha', 'UPI', 'Pune'),
34  (5, 'Amit', 'Cash', 'Delhi'),
35  (6, 'Rohan', 'Net Banking', 'Chennai'),
36  (7, 'Anjali', 'UPI', 'Hyderabad'),
37  (8, 'Kiran', 'Credit Card', 'Mysuru');
38  select * from paymee;
39
40  -- inner join
41  select *
42  from stude as s
43  inner join paymee as p
44  on s.name = p.name;
45
46  -- left join
47  select *
48  from stude as s
49  left join paymee as p
50  on s.name = p.name;
51
```

Output | Result #1

roll_no	name	marks	grade	city	customer_id	name	mode	city
1	Adarsh	85	A	Belagavi	1	Adarsh	UPI	Belagavi
2	Rahul	72	B	Bengaluru	2	Rahul	Credit Card	Bengaluru
3	Sneha	91	A	Mysuru	4	Sneha	UPI	Pune
4	Amit	65	C	Hubballi	5	Amit	Cash	Delhi
5	Priya	88	A	Dharwad	3	Priya	Debit Card	Mumbai
6	Karan	76	B	Belagavi	NULL	NULL	NULL	NULL
7	Anjali	95	A	Bengaluru	7	Anjali	UPI	Hyderabad
8	Rohit	59	C	Mysuru	NULL	NULL	NULL	NULL

Q13 RIGHT JOIN – Retrieve Matching Records Without NULL Student Values

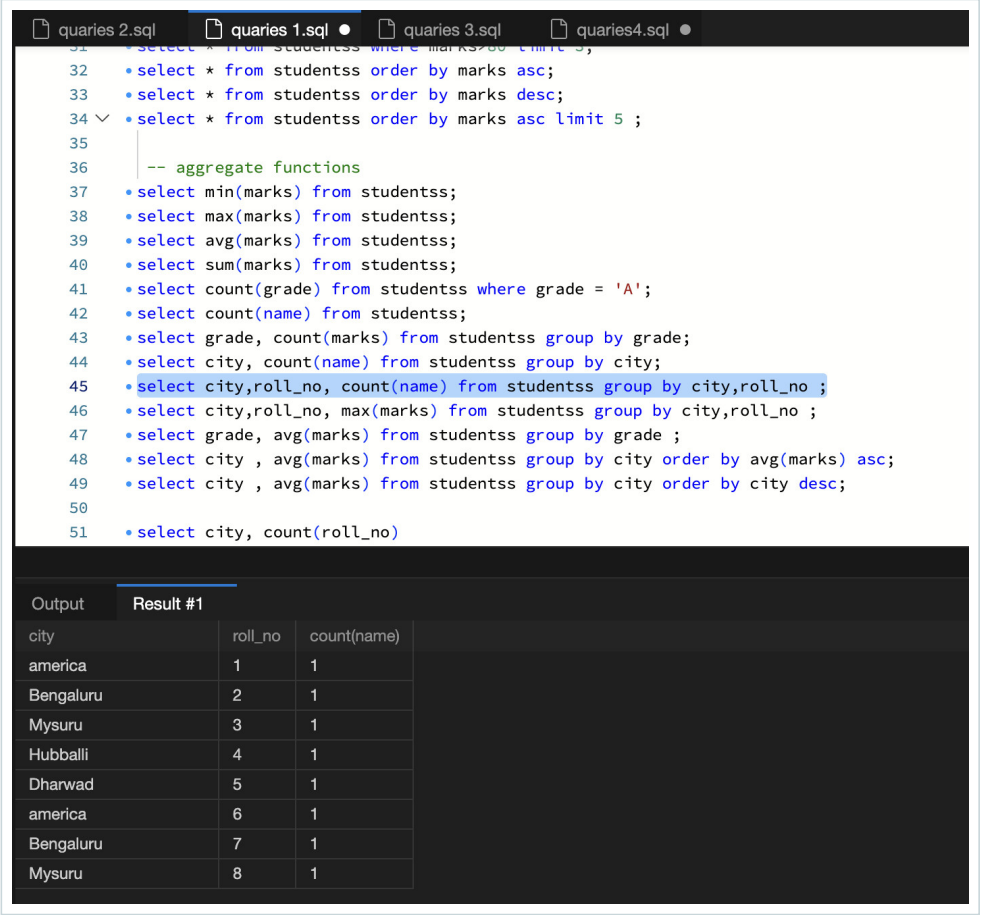
Joins the student and payment tables on roll number and customer ID, keeping only rows where the student roll number is not NULL.



roll_no	name	marks	grade	city	customer_id	name	mode	city
1	Adarsh	85	A	Belagavi	1	Adarsh	UPI	Belagavi
2	Rahul	72	B	Bengaluru	2	Rahul	Credit Card	Bengaluru
3	Sneha	91	A	Mysuru	3	Priya	Debit Card	Mumbai
4	Amit	65	C	Hubballi	4	Sneha	UPI	Pune
5	Priya	88	A	Dharwad	5	Amit	Cash	Delhi
6	Karan	76	B	Belagavi	6	Rohan	Net Banking	Chennai
7	Anjali	95	A	Bengaluru	7	Anjali	UPI	Hyderabad
8	Rohit	59	C	Mysuru	8	Kiran	Credit Card	Mysuru

Q14 Count the Number of Students Grouped by City and Roll Number

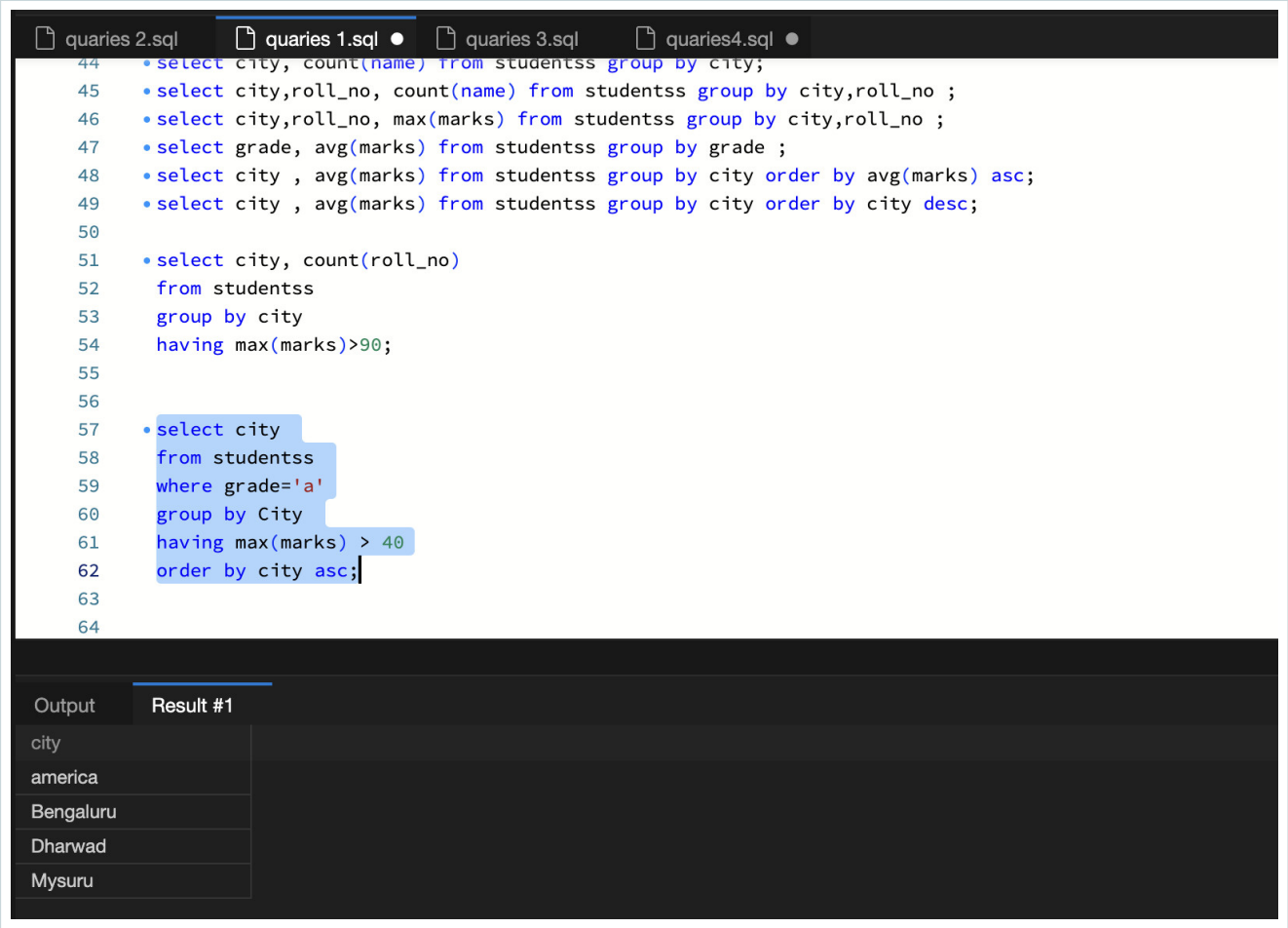
Counts the students grouped by city and roll number.



city	roll_no	count(name)
america	1	1
Bengaluru	2	1
Mysuru	3	1
Hubballi	4	1
Dharwad	5	1
america	6	1
Bengaluru	7	1
Mysuru	8	1

Q15 Cities with Grade A Students Whose Maximum Marks Are Greater Than 40

Displays the cities in ascending order.



2 Linux Practice

Run each command in your Linux terminal or training shell. Some commands may display different values depending on the environment. Capture the command and its real output.

Command Reference

No.	Task	Command	Description
1	Check current directory	`pwd`	Prints the current working directory.
2	List files	`ls`	Lists visible files and folders.
3	List files with details	`ls -l`	Shows permissions, owner, size and modification time.
4	Show hidden files	`ls -la`	Includes hidden entries.
5	Check logged-in user	`whoami`	Prints the current username.
6	Show current date and time	`date`	Displays the system date and time.
7	Create a directory	`mkdir aws_practice`	Creates a folder named aws_practice.
8	Enter the directory	`cd aws_practice`	Changes into the practice directory.
9	Create an empty file	`touch notes.txt`	Creates notes.txt if it does not exist.
10	Write text to a file	`echo 'AWS Glue practice' > notes.txt`	Writes the text into notes.txt.
11	Read a file	`cat notes.txt`	Prints the file contents.
12	Copy a file	`cp notes.txt notes_backup.txt`	Creates a copy of the file.
13	Rename a file	`mv notes_backup.txt backup.txt`	Renames or moves the file.
14	View file details	`stat notes.txt`	Shows file metadata.
15	Check Linux/kernel version	`uname -a`	Displays kernel and system information.

Terminal Output

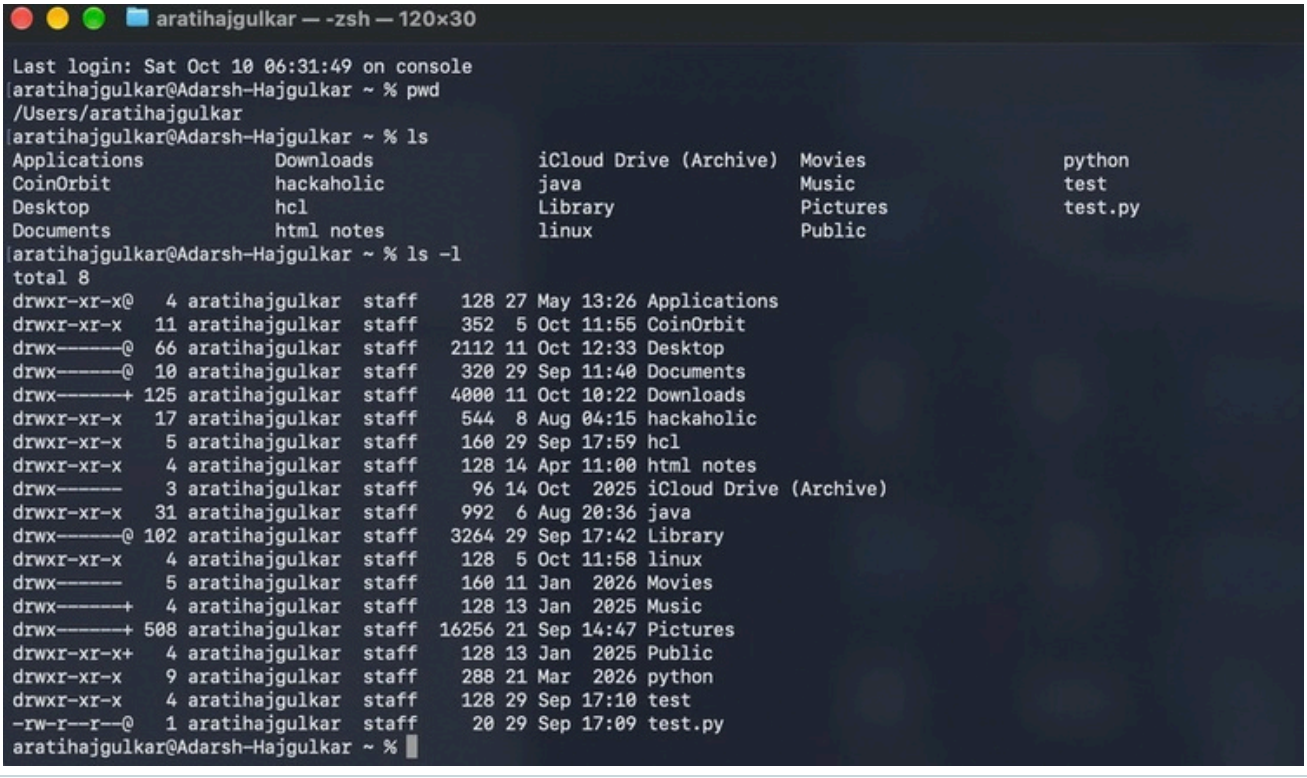


Figure 1: Commands 1–3 – pwd, ls and ls -l

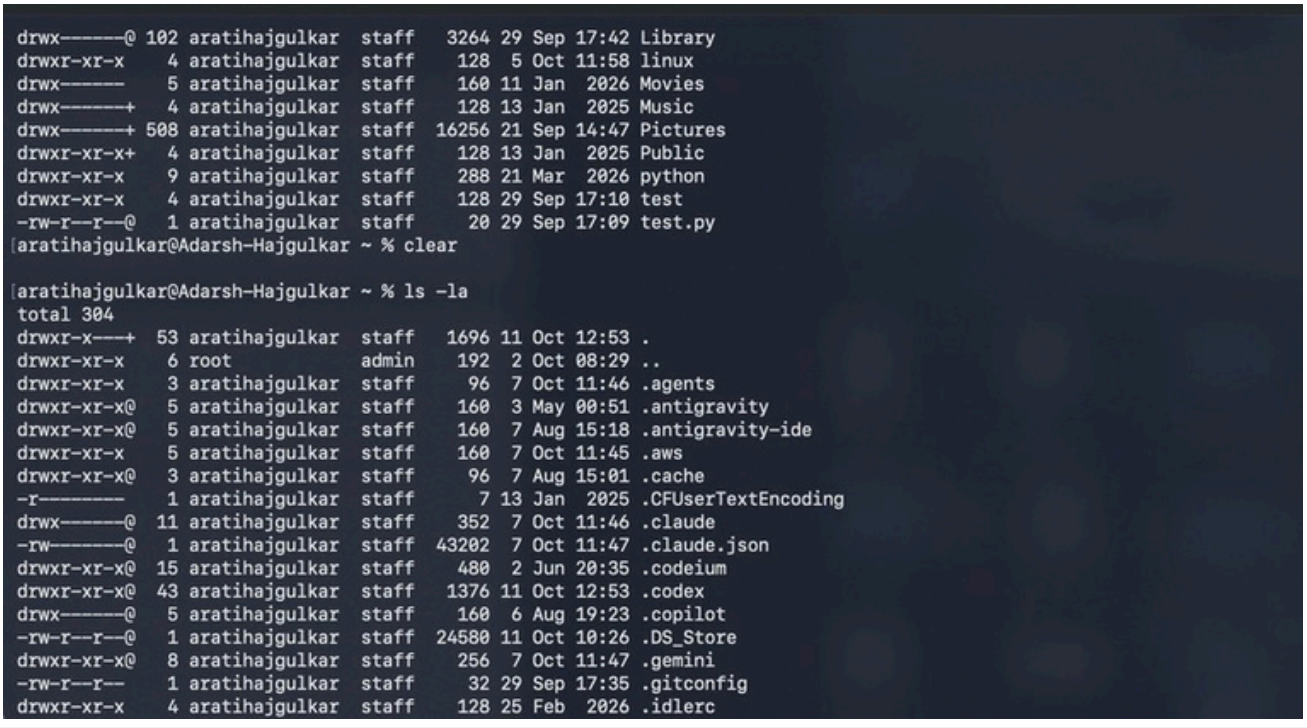


Figure 2: Commands 3–4 – ls -l (continued) and ls -la

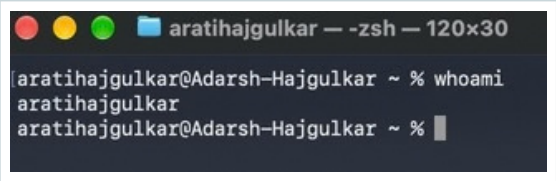


Figure 3: Command 5 – whoami

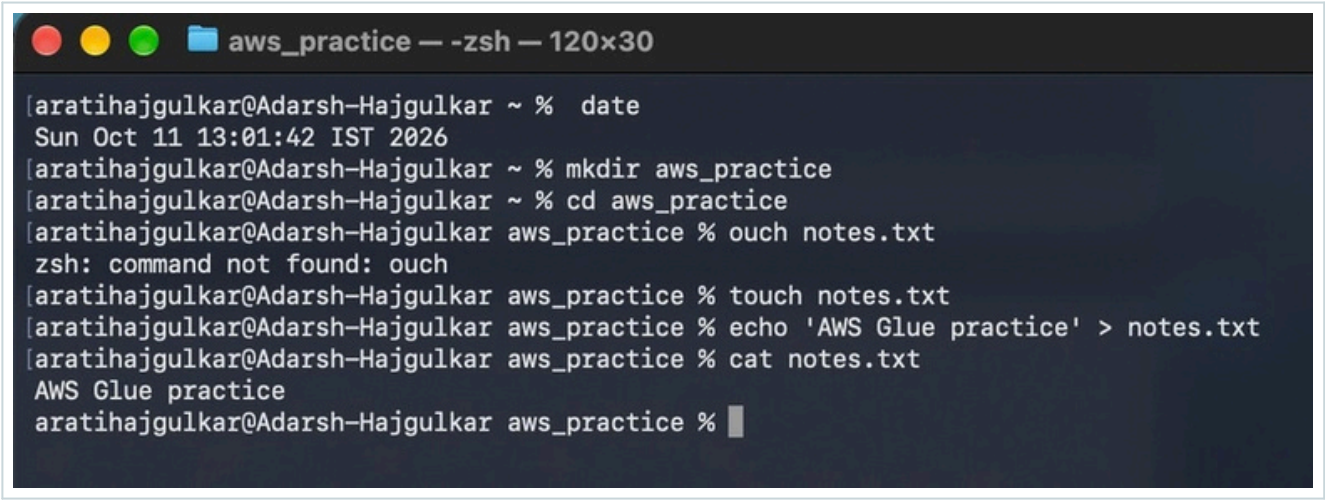


Figure 4: Commands 6–11 – date, mkdir, cd, touch, echo and cat

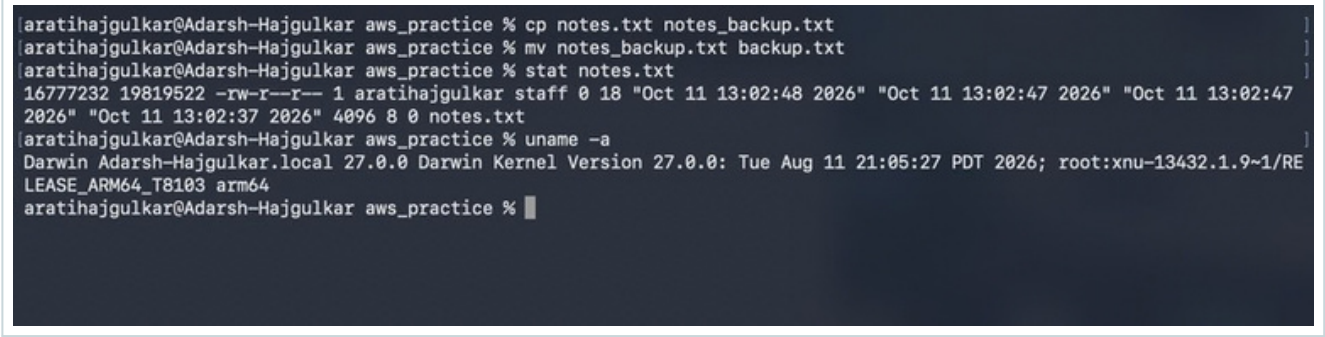


Figure 5: Commands 12–15 – cp, mv, stat and uname -a

3 AWS Glue ETL Pipeline

S3, IAM and AWS Glue workflow. This section of the group project was carried out by Anmol Janamatti (2JI23CS022).

1 AWS Account and Region

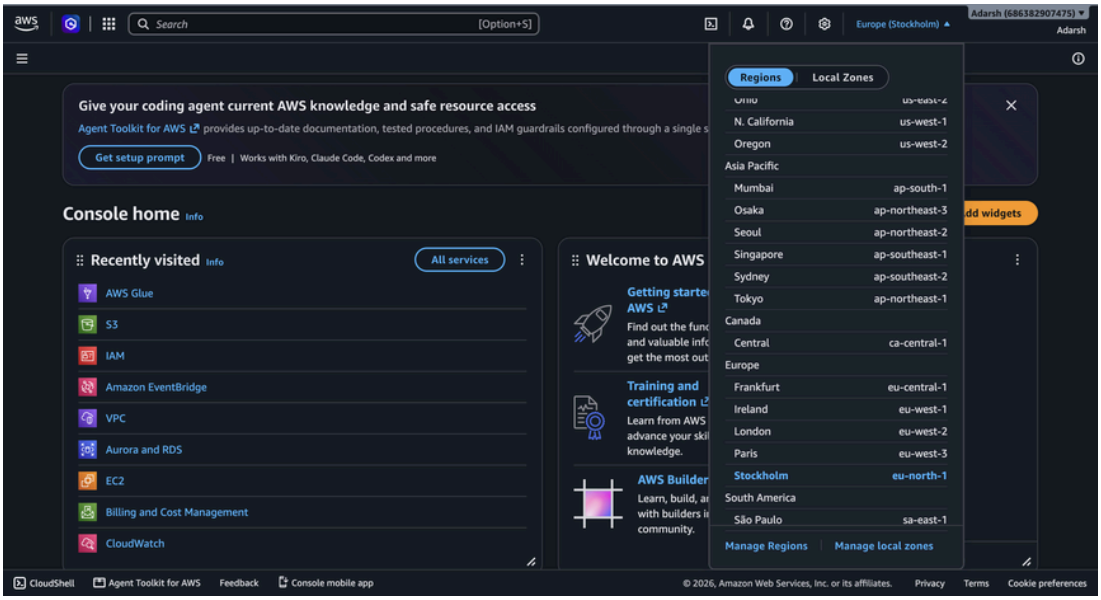


Figure 1: AWS Console Home with the selected region Europe (Stockholm)

2 Amazon S3 Bucket

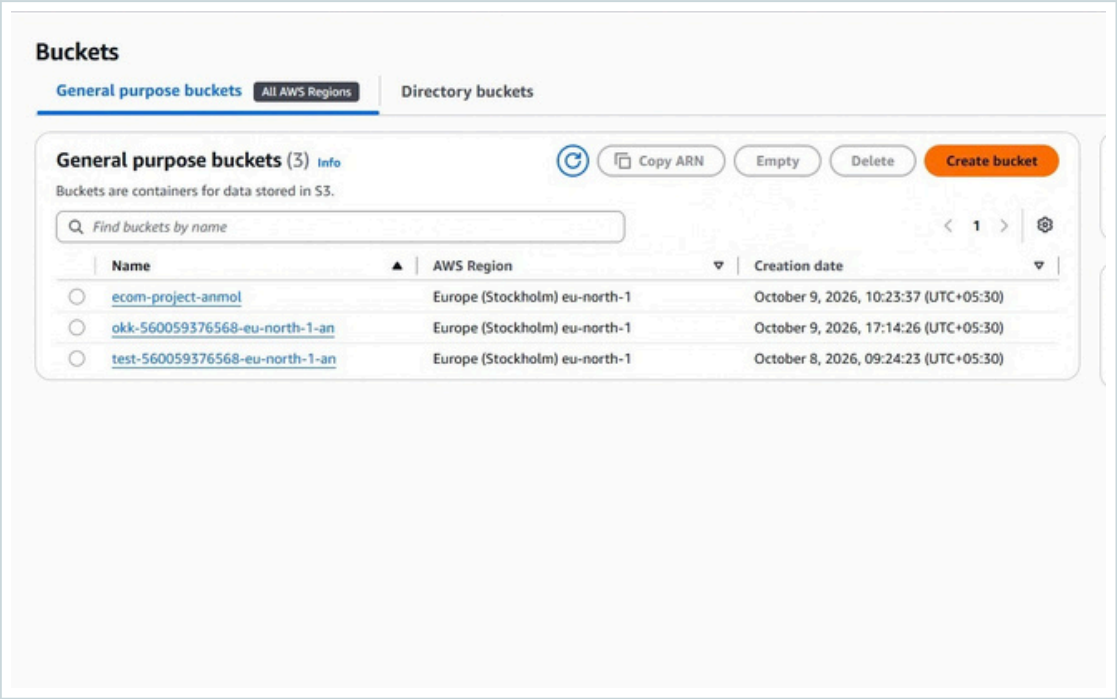


Figure 2: Buckets page of Amazon S3 (Europe (Stockholm) eu-north-1)

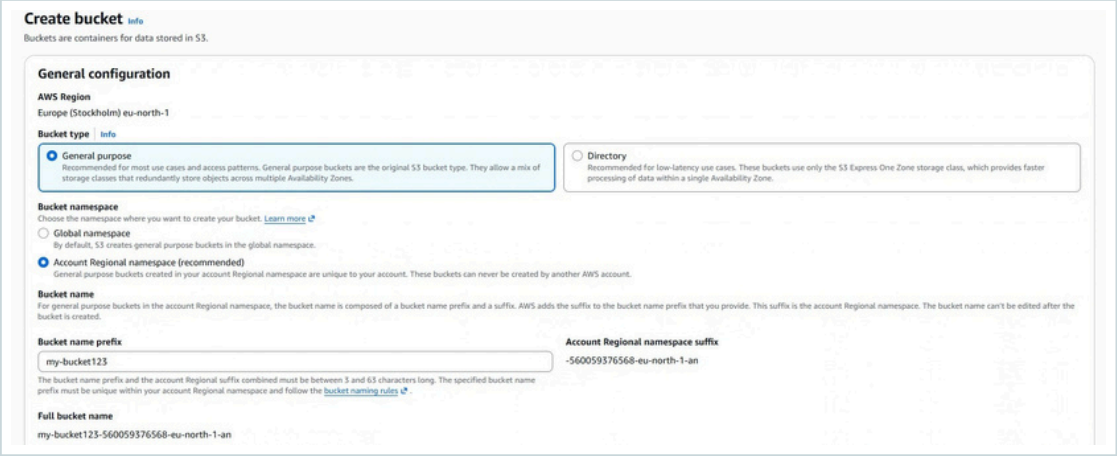


Figure 3: Create bucket: general purpose bucket in eu-north-1 with the bucket name my-bucket123-560059376568-eu-north-1-an

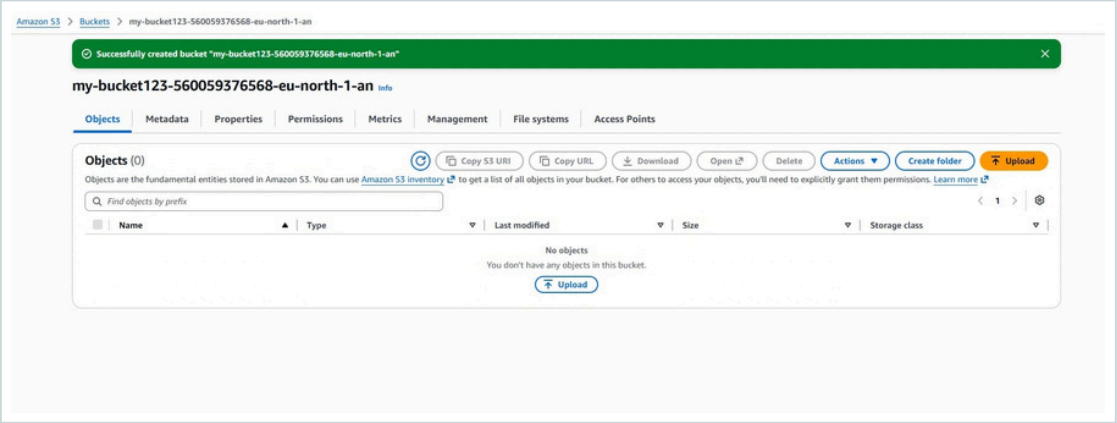


Figure 4: Bucket is created successfully and is empty with 0 objects

3 IAM Role for AWS Glue

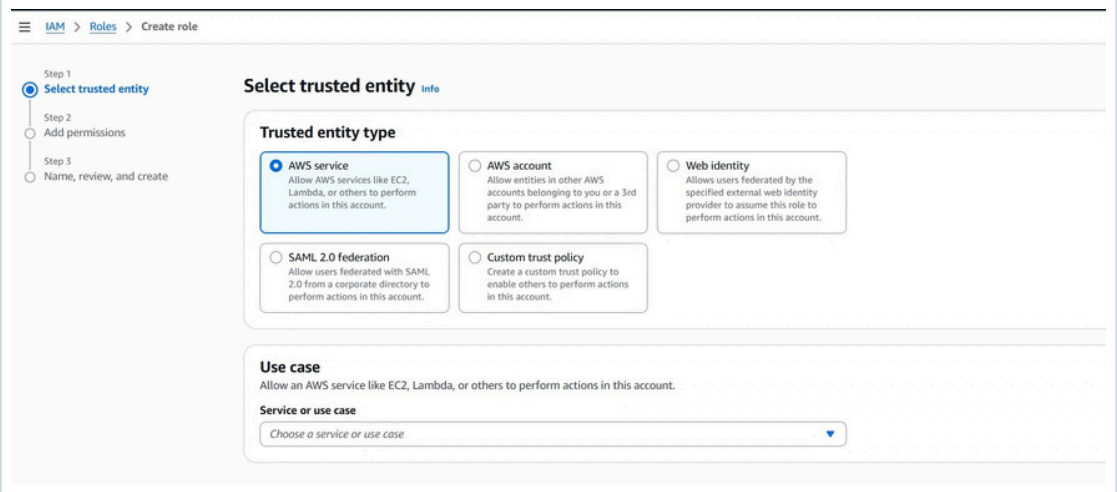


Figure 5: Select trusted entity as AWS service

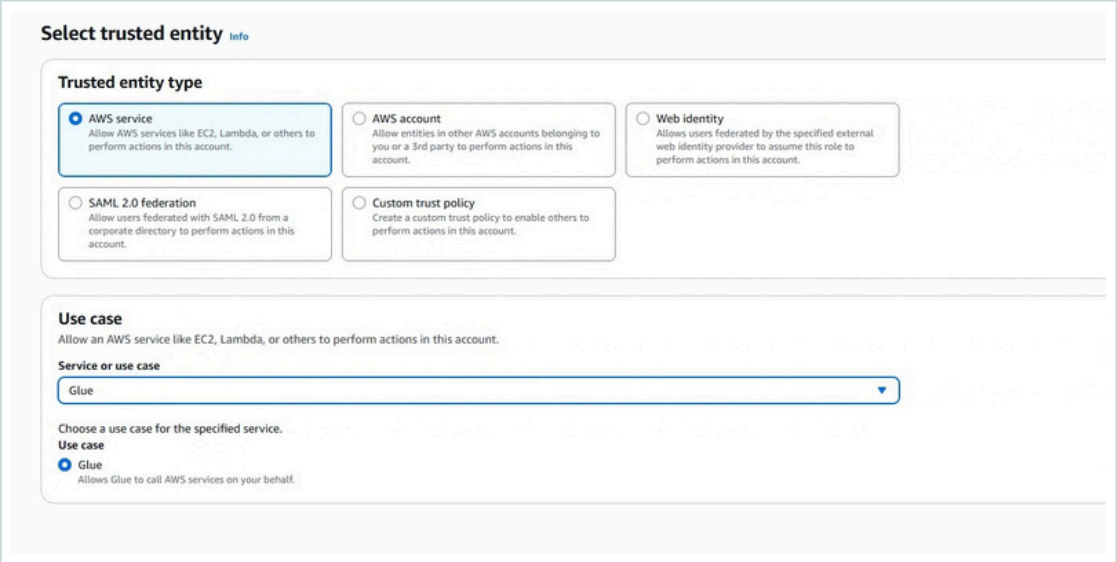


Figure 6: Use case selected as Glue

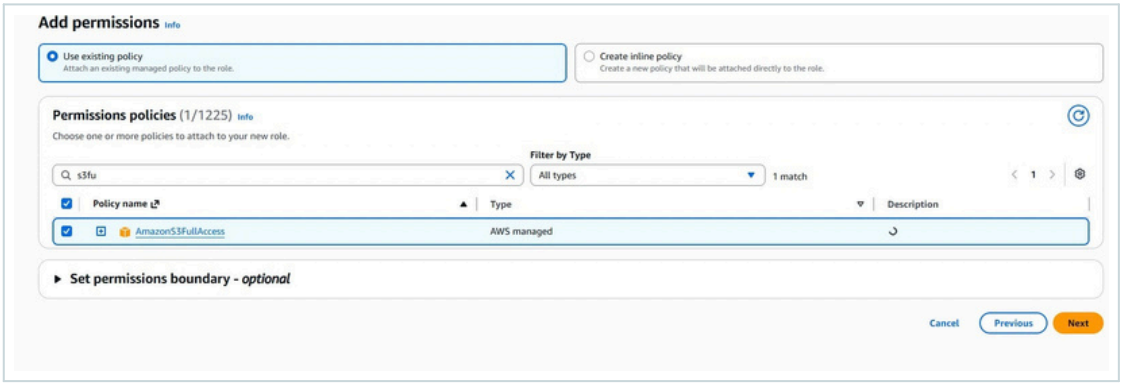


Figure 7: Add permissions: AmazonS3FullAccess policy selected

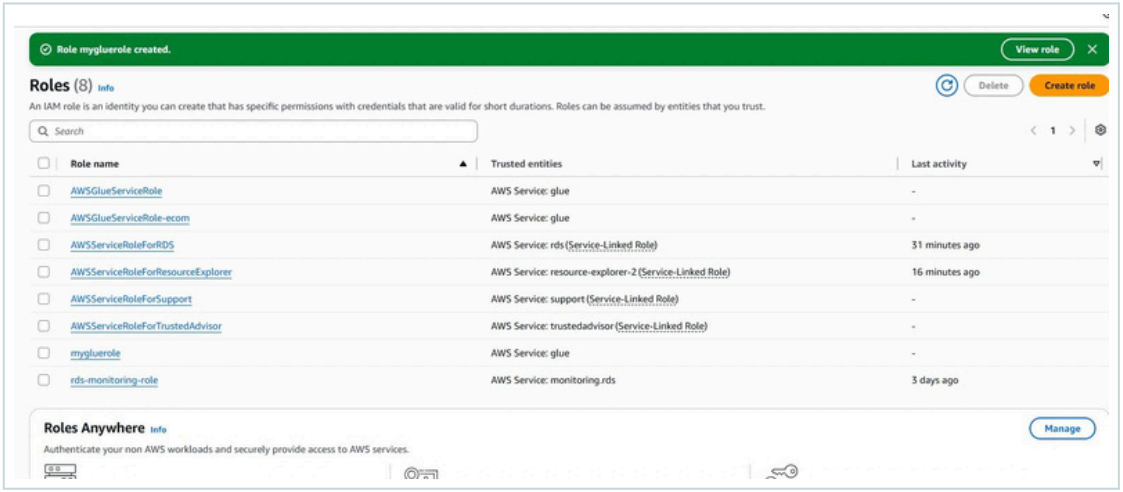


Figure 8: Role myglueerole created

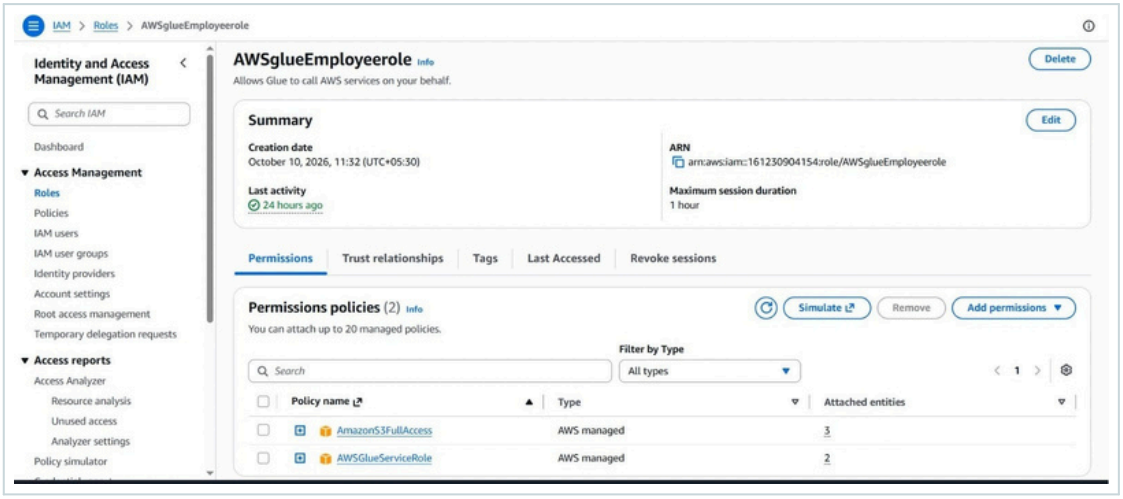


Figure 9: IAM role used for AWS Glue: AWSglueEmployeerole with the AmazonS3FullAccess and AWSGlueServiceRole policies

4 AWS Glue Database

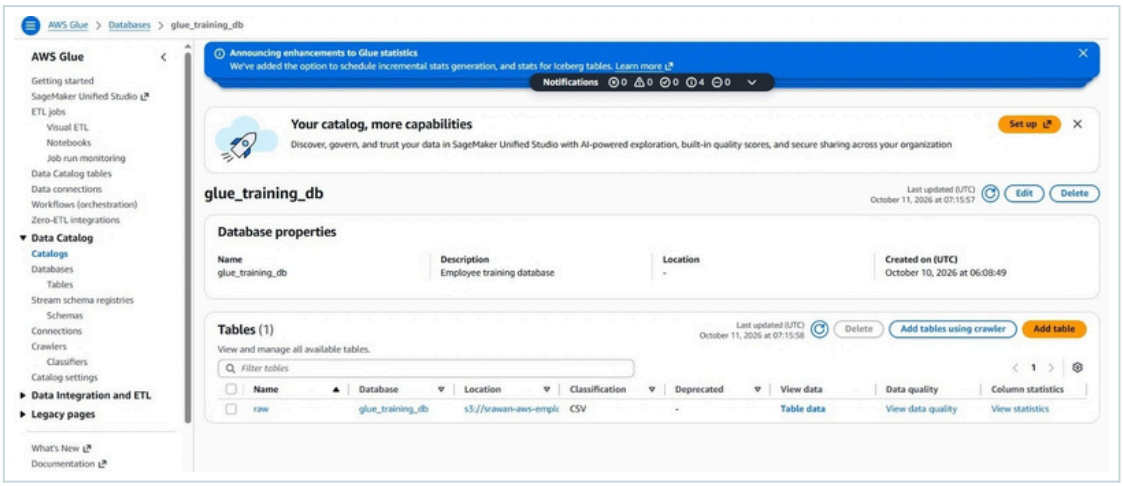


Figure 10: AWS Glue Data Catalog: database glue_training_db with its properties and tables

5 Glue Crawler

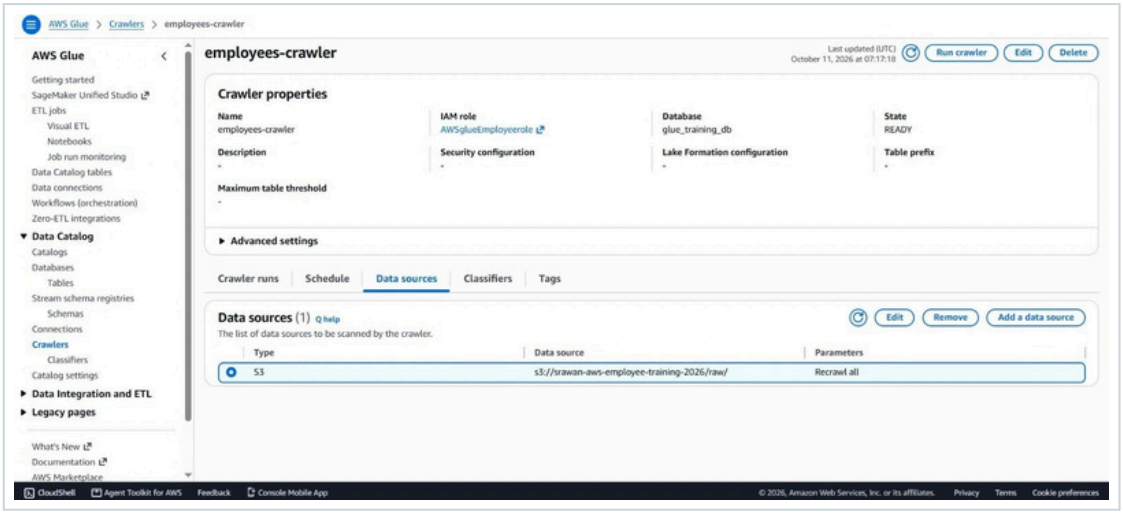


Figure 11: Crawler employees-crawler: S3 data source (raw/ folder), IAM role AWSglueEmployeerole and target database glue_training_db

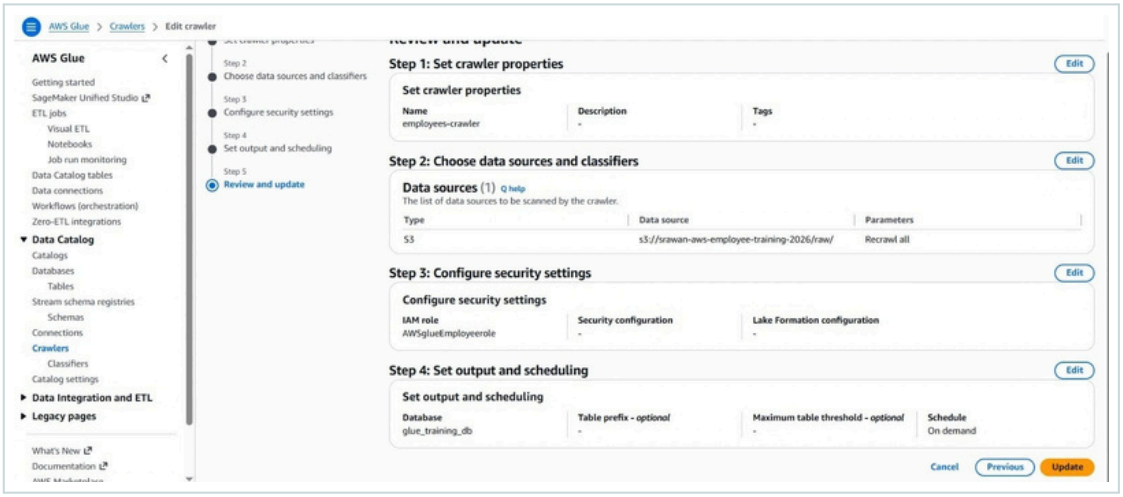


Figure 12: Crawler configuration: review of the crawler properties, data source, security settings and output (schedule: on demand)

6 Crawler Execution

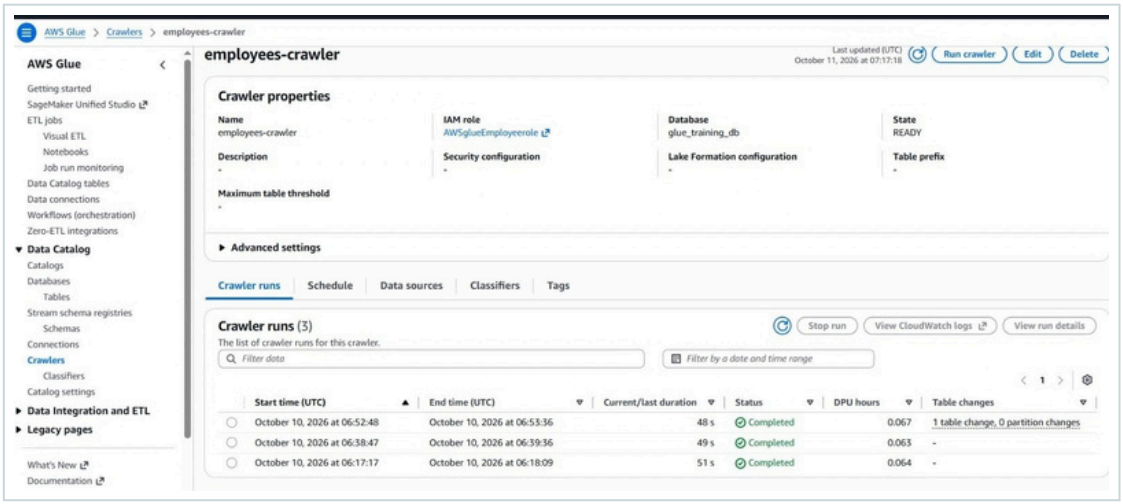


Figure 13: Crawler execution: the crawler runs show the status Completed and the crawler state is READY

7 Glue Data Catalog

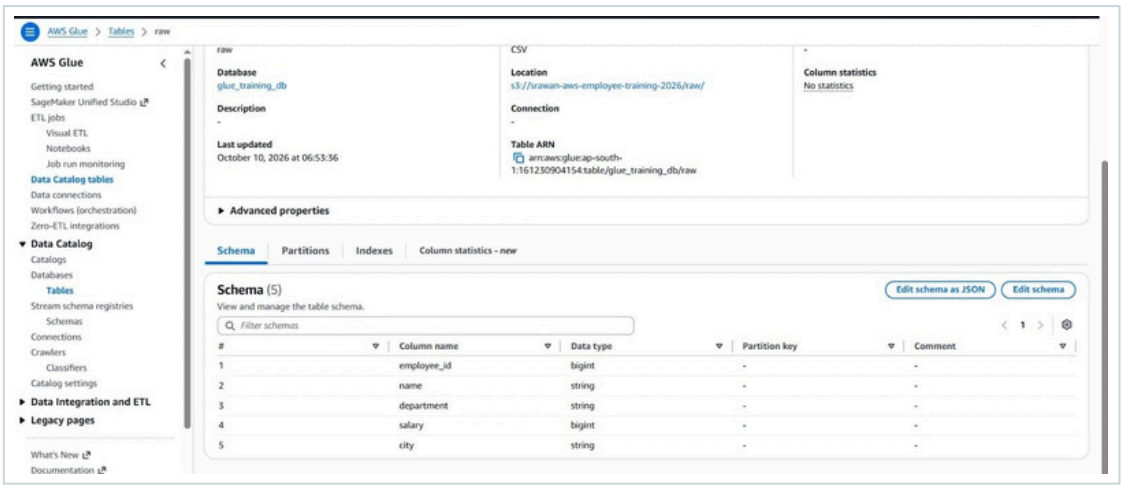


Figure 14: Table raw created by the crawler in glue_training_db, with its schema and data types

8 Glue Visual ETL

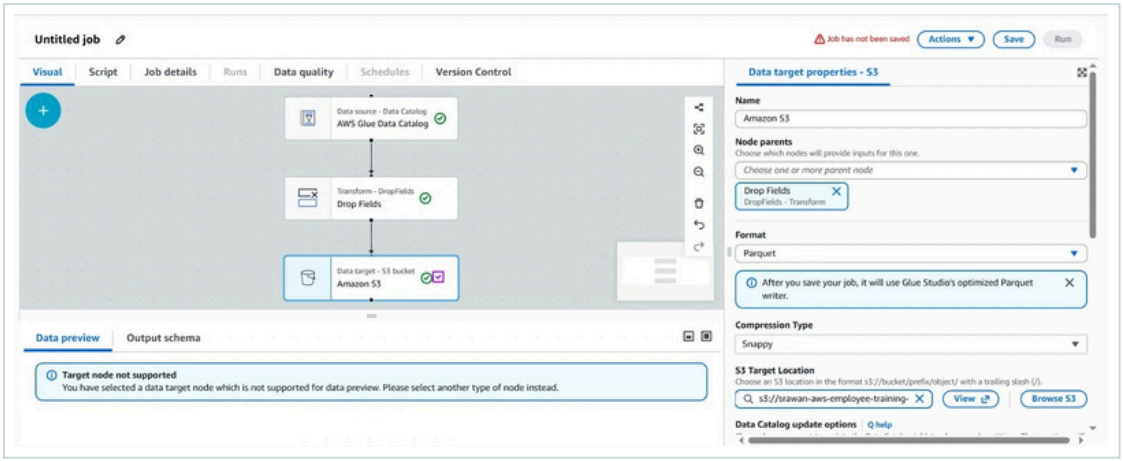


Figure 15: Visual ETL job: Data Catalog source, Drop Fields transform and Amazon S3 target

9 Glue Transformation

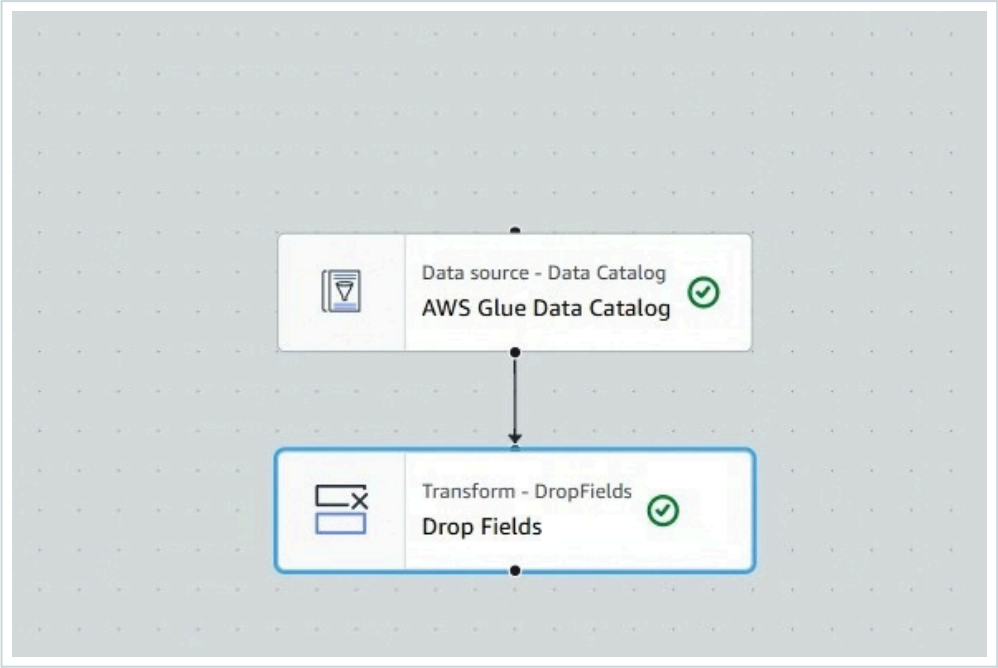


Figure 16: Transformation: Drop Fields node connected to the AWS Glue Data Catalog source

10 S3 Target Configuration

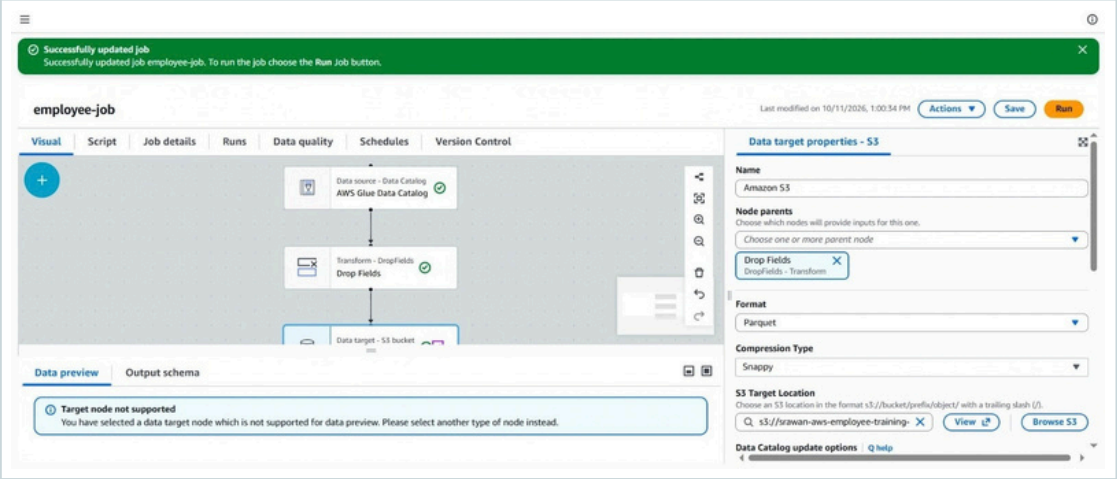


Figure 17: S3 target configuration: Parquet format with Snappy compression and the S3 target location

11 Glue ETL Job Execution

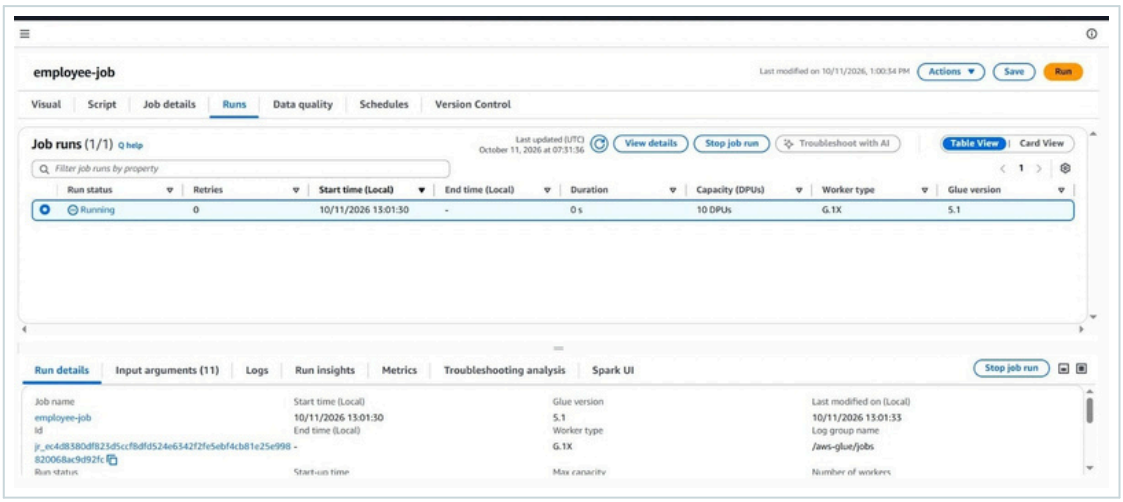


Figure 18: Job employee-job started: run status Running

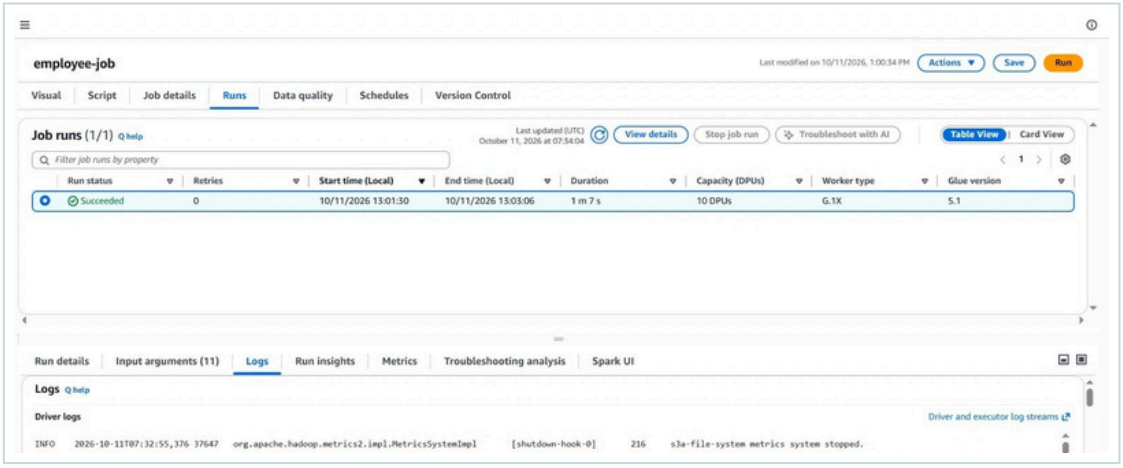


Figure 19: Job employee-job: run status Succeeded

12 Final S3 Output

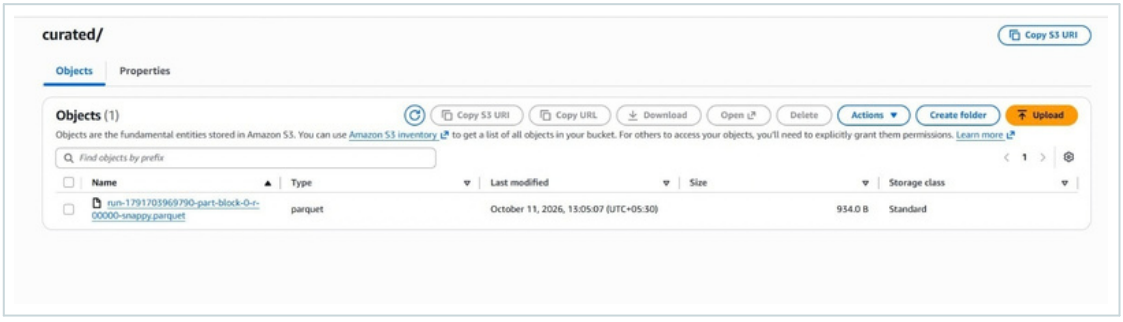


Figure 20: Output in the curated/ folder: a Parquet file generated by the job

Final Explanation: AWS Glue ETL Workflow

1. Amazon S3

Amazon S3 is the storage layer of this workflow. We use a bucket with a raw folder and a curated folder. The employee CSV file is uploaded into the raw folder. The raw folder keeps the original data as it is. The curated folder is kept for the cleaned output.

2. Glue Crawler

A Glue crawler scans the data stored in S3. Our crawler reads the raw folder using an IAM role. It identifies the file format and the column names. It also detects the data type of each column. These details are written to the Glue Data Catalog.

3. Glue Data Catalog

The Data Catalog stores details about the data, not the data itself. It holds the database glue_training_db. The crawler creates a table inside this database. Its columns are employee_id, name, department, salary and city. Glue uses this table as the source of the ETL job.

4. Glue Visual ETL

Glue Visual ETL is used to build the job by connecting boxes. The source box reads the table from the Data Catalog. The next box transforms the data. The last box writes the result to S3. The job is saved as employee-job and needs no hand-written code.

5. Transformation

Drop Fields is the transformation used in this job. It removes the city column, which is not needed in the output. The remaining columns move to the next box unchanged. This gives a smaller and cleaner dataset. The cleaned data is then sent to the S3 target.

6. S3 Curated Data

The S3 target box writes the output to the curated folder. The output format is Parquet with Snappy compression. When employee-job runs, the status shows Succeeded. A Parquet file is created in the curated folder. This file can be used for further analysis and reporting.

Workflow

